ARM PrimeCell

Static Memory Controller (PL092)

Design Manual

ARM PrimeCell Static Memory Controller (PL092) Design Manual

Proprietary notice

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com/

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

1.1.1 Current status and anticipated changes

This document is a draft and is ready for review. Changes are anticipated.

1.1.2 Change history

Issue	Date	Change
A	14 May, 2001	First Draft
B	31st January, 2003	Updated for release r1p2
C	19th May, 2003	Updated for release r1p3

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM DDI 0203C	AHB Static Memory Controller TRM
2	ARM IHI0011	AMBA Specification Rev2.0
3	PL092_INTM_0000_A00	ARM PrimeCell Static Memory Controller (PL092) Integration Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
AHB	Advanced High-performance Bus
ATPG	Automatic Test Pattern Generation
EBI	External Bus Interface
IP	Intellectual Property
RTL	Register Transfer Level
SMC	Static Memory Controller
ASSP	Application Specific Standard Product
DBI	external Data Bus Interface

SmcCore	Static Memory Controller Core block
TIC	Test Interface Controller

1.4 Conventions used in this document

1.4.1 Signals

- Signal names are shown in bold font type.
- Active low signals are prefixed by a lower case ‘n’. In the case of AHB signals by ‘Hn’, or postfixed by ‘n’.
- AHB signals are prefixed by an upper case ‘H’.
- When a signal is described as being ‘Asserted’, the actual level depends on whether the signal is active HIGH or active LOW. ‘Asserted’ means HIGH for active high signals and LOW for active low signals.

1.4.2 Bytes, Half-words and Words

- A byte is 8 bits.
- A half-word is two bytes i.e. 16 bits.
- A word is 4 bytes i.e. 32 bits.

1.4.3 Bits, bytes, K and M

- The suffix ‘B’ (upper case) is used to indicate BYTES.
- The suffix ‘b’ (lower case) is used to indicate BITS.
- When used to indicate an amount of memory, the suffix ‘k’ means 1024.
- When used to indicate a frequency, the suffix ‘k’ means 1000.
- When used to indicate an amount of memory, the suffix ‘M’ means 1024^2 = 1048576.
- When used to indicate a frequency, the suffix ‘M’ means 1,000,000.

2 SCOPE

This document describes the design and implementation of a high performance ARM PrimeCell Static Memory Controller [SMC_PL092]. The design implementation detail is covered in the sections – Section 6 and Section 7. The explanations regarding testbenches and the test programs for functional verification are covered in the Sections 8, 9 and 10.

Section 4 presents an overview of the SMC design and its features. Section 5 enumerates the main blocks on the top level. The detailed explanation of the SmcCore block design with its functional sub-partitioning and the signal interconnections are captured in Section 6. The implementation details of the other major top level blocks of the SMC namely, the TIC and the DBI are described in Section 7. The Section 9 describes the verification strategy with testbench for the verification of the SmcCore and DBI block, while Section 10 describes verification strategy for the TIC block. The test programs used to generate test vectors for functional verification of the SmcCore, DBI and the TIC are detailed in Section 11.

3 INTRODUCTION

The Static Memory Controller (SMC) is an Advanced Microcontroller Bus Architecture (AMBA) slave block that connects to the Advanced High-performance Bus (AHB).

Memory system bandwidth is typically one of the most important determinants of ASIC/ASSP system performance and is typically heavily customised for each application. However, as controlling high performance memory systems becomes more complex, the amount of time and resource required to implement a customised solution becomes prohibitive. The ARM Static Memory Controller aims to help designers integrate high performance, low power static memory interface into a design as quickly as possible.

The ARM PrimeCell Static Memory Controller is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1].

The AHB related specification can be found in the AMBA Specification [Ref. 2].

4 STATIC MEMORY CONTROLLER DESIGN OVERVIEW

4.1 Features

This ARM PrimeCell Static Memory Controller offers the following features:

- Soft macro block available in VHDL and Verilog RTL.
- Fully scan insertable design.
- Functional verification using ARM BusTalk functional test environment.
- Compatible with AMBA AHB on chip bus systems.
- Support for external asynchronous wait control.
- Polarity of the external wait input signal programmable.
- Chip select polarity for each bank programmable independently.
- Minimises switching of the unused byte lanes of the external memory data bus.
- Configurable size at reset for boot memory bank seven using external control pins.
- Support for an additional memory controller interface.
- Dynamic priority arbitration between the memory controllers, for the external bus control

The ARM PrimeCell Static Memory Controller supports:

- Static memory mapped devices including RAM, ROM, flash memory and burst ROM.
- Asynchronous page mode read operation in non-clocked memory sub-systems.
- Asynchronous burst mode read access to burst mode ROM devices.
- 8-, 16- or 32-bit wide external memory data paths.

- Little-Endian and Big-Endian system architectures.
- Independent configuration for up to eight memory banks, with a capacity of 64 Mbytes.
- Programmable access wait states (0 to 31 cycles) for burst reads from Burst ROM devices
- Programmable access wait states (0 to 31 cycles) in case of normal writes and reads
- Programmable bus turn-around cycles (0 to 15).
- Write enable and byte lane select outputs for use with 32, 16 or 8-bit SRAM devices with independent byte lane control for each memory bank.
- Delayed Output Enable assertion during reads from memory devices with the delay value being programmable.
- Delayed Write Enable assertion during writes to memory devices with the delay value being programmable.
- Additional support for interfacing with an external bus arbiter, like the EBI peripheral in place of the internal bus interface block.

For further details about the function and the programmer's model, please refer to the ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1].

5 SMC PRIMECELL PERIPHERAL BLOCK

There are three main functional blocks in the SMC PrimeCell :

- Static Memory Controller Core (SmcCore) – This block processes the read and write transfer requests to the external memory device
- Test Interface Controller (TIC) - used during testing to read external test vectors and apply them to the system through the AMBA AHB master interface.
- Data Bus Interface (DBI) - This block arbitrates between the SmcCore, TIC and the additional memory controller for the control of External bus. Bypass logic is also implemented in the DBI which will enable the Smc to interface with an external simple bus arbiter, like EbiSdram.

A block schematic of the ARM PrimeCell Static Memory Controller is shown in ***Figure 5-1.***

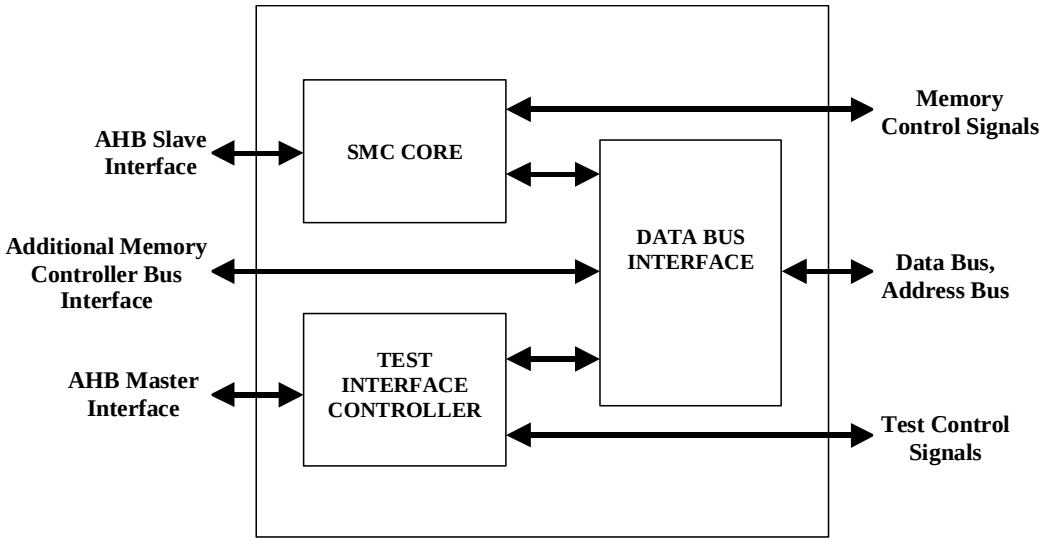

Figure 5-1: ARM PrimeCell Static Memory Controller block diagram

The implementation descriptions of each of the functional block are provided in the following sections.

6 SMCCORE SUB-BLOCK PARTITIONING AND INTERNAL MODULE IMPLEMENTATION DETAILS

There are three functional sub-modules within the SmcCore block. These are:

- AHB Interface - used to sample the control signals from the AHB, and drive the AHB read data and response signals at the end of a transfer. Contains all the bank configuration registers, parameter registers and status registers.
- Transfer Control - used to control the flow of the read and write transfers between the AHB and the external bus. Consists of a main control state machine, access timing and delay timing counter blocks and external wait control logic.
- External Interface Block - used to perform transfers on the external bus, by driving the external control signals. Constitutes external interface logic block, synchroniser block and write enable control signal block.

The operation of these blocks is described in the following sections. A block schematic of the ARM PrimeCell Static Memory Controller core is shown in ***Figure 6-1***

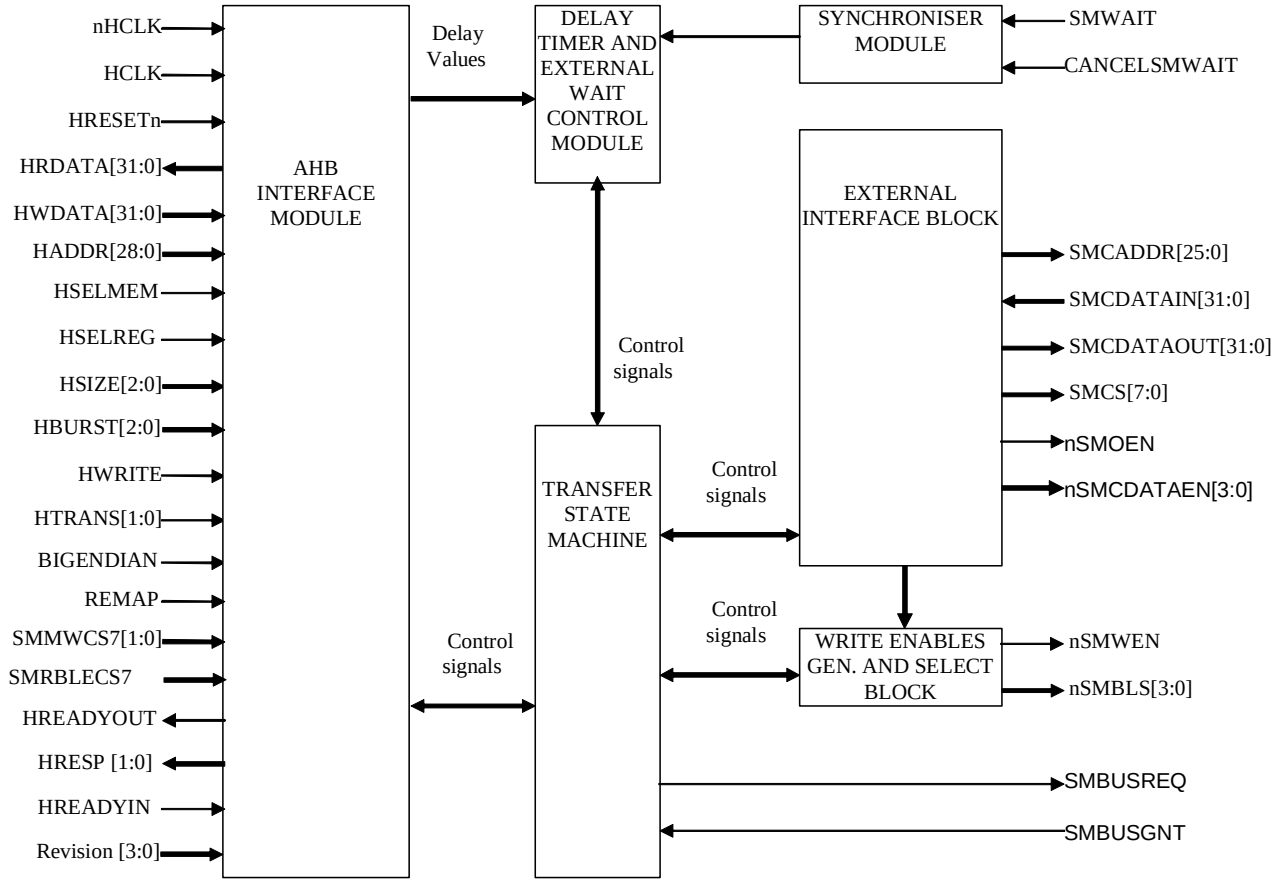

Figure 6-1: ARM PrimeCell Static Memory Controller Core block diagram.

6.1 AHB Interface Block

This block provides the interface to the AMBA AHB bus. It consists of the AHB response generation logic, all of the configuration control and status registers for the external memory banks, the PrimeCell and Peripheral Identification registers. There are three main parts to this block, which are described in the following sections.

6.1.1 AHB inputs

The AHB input signals are sampled on the rising edge of the clock when **HREADYIN** is asserted HIGH and the SMC peripheral is selected. These signals are then used in the rest of the system to generate the External Interface Block control signals, or are used to perform accesses to the internal registers.

The **REMAP** input is used to control the memory bank selection for banks zero and seven while the system is booting after reset, and is registered to generate the **RemapReg** output. This is passed directly to the External Bus Interface block.

The **BIGENDIAN** input is used to configure the system for Big-Endian or Little-Endian operation, and is assumed to be a static input pin. The SmcCore does not support dynamic Endianness. This signal is used in the External Bus Interface block to control the routing of data to the proper byte lanes used during data transfers.

The **SMMWCS7[1:0]** inputs are used to configure the size for memory bank seven immediately after reset. The inputs are sampled on the rising edge of the reset (so that they are only captured once), and are then used to set the relevant bits of the bank seven control register.

The **SMRBLECS7** input is used to configure the RBLE pin immediately after reset. This input is sampled on the rising edge of the reset(so that it is captured once), and are then used to set the RBLE bit of the bank seven control register.

Control signals **MemRdReq, MemWrReq, WtdRdReq, and WtdWrReq** for the memory read and write transfers are decoded and sent to the transfer state machine. These signals will only be asserted for non-sequential and sequential transfers to external memory of width less than or equal to 32-bits. When a new read or write transfer is initiated, the **MemRdReq** or **MemWrReq** signal is generated and sent out to other blocks to proceed further. Along with this **HAddrCrnt, HTransCrnt, HSizeCrnt, HBurstCrnt** signals are generated which represents the address, transfer type, transfer size, burst information for the current transfer. For normal write transfers [non external wait controlled mode], the transfer completion response with **HREADYOUT** = HIGH and **HRESP** = OKAY is given with zero wait cycles immaterial of the access count value programmed. The **HREADYOUT** is pulled LOW in case of read transfer initiation from the AHB and would be pulled HIGH once the read transfer is complete.

Waited transfers are performed in case when a read or write transfer is initiated while a write transfer or **MW** bits programming of SMBCR7 register is going on. In this case **WtdWrReq** or **WtdRdReq** signal is produced and sent out. These signals are asserted till the current write transfer or the **MW** programming is over. The address, transfer type, transfer size, burst information, Bank address of the waited transfer are stored in the **HAddrWtd, HTransWtd, HSizeWtd, HBurstWtd, BankAddrWtd** registers.

Depending on which of the above type of transfers is being serviced, one of the two sets of the registered control signals, listed above, are multiplexed on to **HTransRegCo, HSizeRegCo, HBurstRegCo, BnkAddStrCo** lines respectively. These signals are passed on to other blocks at the start of current read or write transfer and while servicing the waited transfer, when the current write transfer or **MW** programming is complete. For both waited read and write transfers the **HREADYOUT** is pulled LOW and would be pulled HIGH once the transfer is complete.

During write transfer for the cases when **HSIZE** < **MSize** a read-write buffer is used to collect the partial data before the external transfer is performed. To indicate that the buffer is filled and external transfer can start, a signal called **BufWrOver** is generated and routed to other blocks. In cases where **HSIZE** = **MSize,** this buffer is bypassed and the **HWDATA** is directly routed to the external data out register **SMDATAOUT**. For this a signal called **BufByPassCo** is generated and routed.

BankCmpCo is the signal, which indicates whether the bank address of the current transfer and the next transfer is same or not. This is used to introduce turn around cycles and to de-assert chip select.

Write protection checking and AHB transfer size error checking are performed in this block.

6.1.2 AHB outputs

The registered AHB slave response outputs **HREADYOUT** and **HRESP** are driven according to the transfers performed to the internal registers and the external memory. Only OKAY and ERROR responses are generated - the SmcCore peripheral will not generate SPLIT or RETRY responses.

WAIT states are inserted :

- during a two cycle error response
- during an external read transfer that has not yet completed
- when an external transfer is being performed and a second external transfer is requested on the AHB
- during an externally waited transfer that has not yet completed (read or write)
- during a valid transfer to the SMC before the bank 7 memory size value has been configured immediately after reset using the SMMWCS7[1:0] pins.
- during a register write so that the parameters get updated in the registers before using them.
- For each individual External Wait controlled read or a write transfer

An ERROR response is provided :

- during a transfer of size greater than 32-bits to external memory (the HSizeErr signal is generated in AHB Interface block)
- if a write transfer is attempted to a Write-Protected Memory device (the WPErr signal is generated in this module).
- after an externally waited transfer has timed out (WaitToutErr signal is generated in Transfer Control block)
- when an AHB master tries to initiate a new transfer in the WaitToutErr case and the SMWAIT input is still in the asserted state due to the previous transfer.

The **WaitToutErr** input from the Timer block, is used to generate ERROR responses when required.

The read data output is generated either from the internal registers, or from the external read data value passed from the External Interface Block, using the registered **HSELREG/HSELSMC** signals to select the source. During reads from external memory of widths less than 32-bit, the data will be routed on to only the relevant byte lanes - it is assumed that the master performing the read transfer will select the correct data byte lanes. The external memory read data value generation is controlled by the External Interface block (EIB), and is a registered output. Also during reads, optimisation is done for transfers with HSIZE < MSize condition, by caching read data. Here after the first read the subsequent data's are given from the internal buffer **RdWrBuf** with zero wait cycles and without any extra memory accesses, if the subsequent read addresses from the AHB fall in the same address range. This is confirmed by comparing the relevant **HADDR** value of the current with the **SmcAddrReg** value. A signal called **AhbRdOver** is generated at the completion of each read operation, when all the data packets have been read by the AHB from the **RdWrBuf** buffer. This signal is sent to the TSM [transfer state machine] block for making decision on the subsequent state transitions.

6.1.3 Internal registers

Further details about the internal registers can be found in the ***ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1]***, ***section 5 Programmer's Model***.

The two **HSELSMC** and **HSELREG** inputs are used to determine if the access is to the external interface or to the internal registers.

Read and write accesses to the internal registers are controlled using the sampled AHB input signals. The address is decoded to select a register bank, and then the control signals are used to perform the transfer. For a write, the write data from **HWDATA** lines are used to update the value in the selected register, and for a read, the register data value selected is used to drive the AHB read data bus **HRDATA**.

In case a register read or write is requested, when MW programming is in progress, the relevant control signals are stored and wait cycles are inserted till the programming is completed.

Only 32-bit transfers should be performed to the internal registers and it is the responsibility of the master to ensure the same. It should be noted that transfers of any other size will still be carried out by using the appropriate lines of the **HWDATA** and **HRDATA** bus. This will happen assuming those lines would have valid data.

The control register values are only programmed through the AHB, and their values cannot be changed any other way. The memory width configuration bits in the bank-7 control register are initially set using the **SMMWCS7[1:0]** input pins after a reset, but these bits may be changed by subsequent writes to the bank-7 control register. The control register values are made available to the Transfer Control and Timer Blocks.

The status register values are set through internal signals, from the AHB Interface block for the bus error, and from the Transfer Control block for external wait timeout and write protect errors. These bits may only be cleared with a write of ‘1’ from the AHB - a write of ‘0’ has no effect.

The peripheral and PrimeCell ID register values are set in the hardware, and are read only registers.

Once the register write is over, all the parameters like **WST1, WST2, WSTOEN, WSTWEN, IDCY, RBLE, BM, MW[1:0], WP, WaitEn, WaitPol, CSPol[7:0]** corresponding to the particular bank selected will be decoded and routed to other blocks. The **MW[1:0]** are further decoded into 3 separate combinational signals relevantly called **MSize08**, **MSize16** and **MSize32**, which are routed to the other blocks for use during valid transfers.

In an External Wait controlled mode of operation when an externally waited transfer times out, an ERROR response is generated and **WaitToutErr** bit is set in the respective status register **SMBSR**. From the point when the wait time-out condition occurs, a continuous check is kept on the status of the **SMWAIT** input line. This status is reflected on the **WaitStatus** bit of the **SMBEWS** register. This register is common to all the external wait controlled banks. The **WaitStatus** bit is asserted from the point when the Wait time-out error occurs and is kept asserted till the **SMWAIT** value of the previous transfer is de-asserted. The master is expected to continuously poll this register bit so as to determine when to initiate the new valid transfer. Otherwise, if the master initiates a new transaction before the de-assertion of the previous **SMWAIT**, then the SmcCore will respond with ERROR response continuously.

6.1.4 Block diagram

The block diagram of the AHB interface is shown in ***Figure 6.1-1***.

The input and output signals of the AHB interface block with a small description for each has been enumerated in the ***Table 6.1-1***.

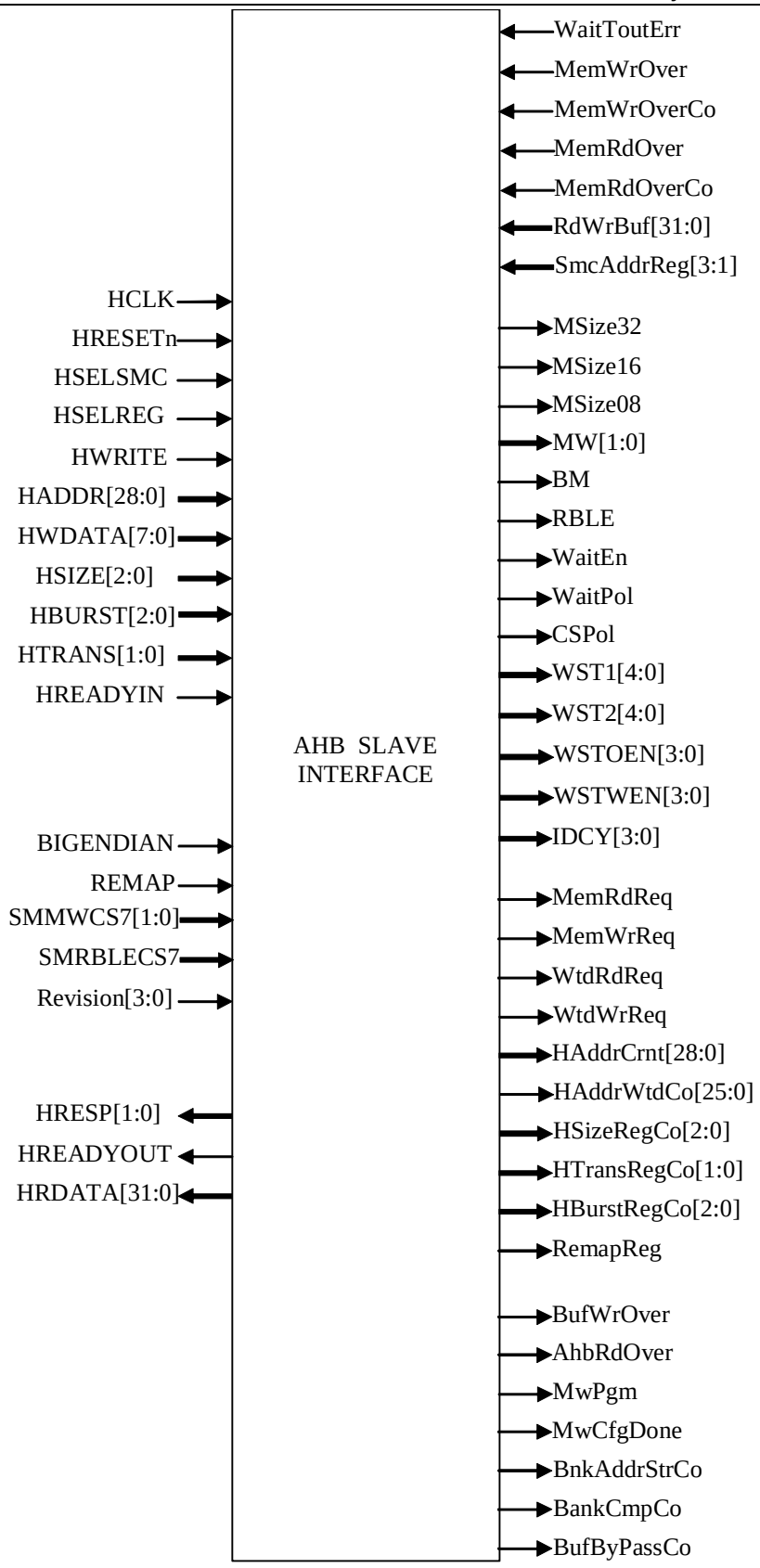

Figure 6.1-1 AHB Interface block

Signal Name	Type	Source/Destination	Description
HCLK	Input	Clock Source	System clock
HRESETn	Input	Reset Controller	System reset
HADDR[28:0]	Input	AHB Master	System address
HTRANS[1:0]	Input	AHB Master	System transfer type
HWRITE	Input	AHB Master	Read/write control
HBURST[2:0]	Input	AHB Master	Burst type
HSIZE[2:0]	Input	AHB Master	System transfer size
HWDATA[8:0]	Input	AHB Master	System write data
HSELSMC	Input	AHB Decoder	External memory select
HSELREG	Input	AHB Decoder	Internal register select
HREADYIN	Input	Other AHB Slaves	System wait input
REMAP	Input	System	System memory map state
SMMWCS7[1:0]	Input	Input Pads	External input which determines the memory width of bank 7
SMRBLECS7	Input	Input Pad	External input which determines the RBLE value to be driven after reset.
Revision[3:0]	Input	RevAnd Block	Revision Number from SmcRevAnd
WaitToutErr	Input	Transfer Control block (DelayTimer)	External wait timeout error
MemWrOver	Input	External Interface Block	Memory device Write completion signal
MemWrOverCo	Input	External Interface Block	Combinational version of MemWrOver
MemRdOver	Input	External Interface Block	Data Read completion from the Memory
MemRdOverCo	Input	External Interface Block	Combinational version of MemRdOver
RdWrBuf[31:0]	Input	External Interface Block	Registered the Read data bus from EIB
SmcAddrReg[3:1]	Input	External Interface Block	Slice of the registered SMCADDR bus
HRDATA[31:0]	Output	AHB Master	System read data
HREADYOUT	Output	AHB Master	System wait output

HRESP[1:0]	Output	AHB Master	System transfer response
HAddrCrnt[28:0]	Output	External Interface Block	Registered address
HAddrWtdCo[25:0]	Output	External Interface Block	Combinational version of HAddrCrnt for waited Memory transfer
HTransRegCo[1:0]	Output	Transfer Control block	Registered transfer type
HSizeRegCo[1:0]	Output	Transfer Control block	Registered transfer size
HBurstRegCo[2:0]	Output	Transfer Control block and External Interface Block	Registered Burst type
RemapReg	Output	External Interface Block	Registered REMAP bit
MemRdReq	Output	Transfer Control block and External Interface Block	Memory read initiation
MemWrReq	Output	Transfer Control block and External Interface Block	Memory write initiation
WtdRdReq	Output	Transfer Control block and External Interface Block	Stored read transfer request
WtdWrReq	Output	Transfer Control block and External Interface Block	Stored write transfer request
WST1[4:0]	Output	Transfer Control block (Delay Timer)	Normal read access value of the currently targeted memory device
WST2[4:0]	Output	Transfer Control block (Delay Timer)	• Write access value in case of SRAMS & Flashes • Faster read access value in case of Burst ROMS
WSTOEN[3:0]	Output	Transfer Control block (Delay Timer)	The OEN assertion delay value of the currently targeted memory device
WSTWEN[3:0]	Output	Transfer Control block (Delay Timer)	The WEN assertion delay value of the currently targeted memory device
IDCY[3:0]	Output	Transfer Control block (Delay Timer)	Turnaround value of the currently targeted memory device
BM	Output	Transfer Control block and External Interface Block	Burst ROM device indication
CSPol[7:0]	Output	External Interface Block	Chip select polarity programmed for each bank
WaitPol	Output	Transfer Control block (Delay Timer)	SMWAIT polarity for the currently targeted memory

			device
WaitEn	Output	Transfer Control block (Delay Timer)	External wait enable indication for the currently targeted memory device
RBLE	Output	External Interface Block	Byte lane enabled device indication
MW[1:0]	Output	Transfer Control block	Memory Width of the currently targeted memory device
MSize08	Output	External Interface Block	Signal to indicate that a 8-Bit memory is targeted
MSize16	Output	External Interface Block	Signal to indicate that a 16-Bit memory is targeted
MSize32	Output	External Interface Block	Signal to indicate that a 32-Bit memory is targeted
BufWrOver	Output	External Interface Block and Transfer Control block	Indication of internal buffer full status during memory writes operation.
AhbRdOver	Output	Transfer Control block	Indicates the completion of read by the AHB.
MwPgm	Output	External Interface Block	Indicates the waited transfer is due to MW bit programming operation
MwCfgDone	Output	Transfer Control block	Indicates the completion of MW bits programming after reset
BnkAddStrCo	Output	External Interface Block	Stored value of the Bank Address
BankCmpCo	Output	Transfer Control block	Indicates the successive transfers are to the same bank
BufByPassCo	Output	External Interface Block and Transfer Control block	Indicates that the HSIZE = MSize and hence the internal **RdWrBuf** can be bypassed

Table 6.1-1 AHB Interface block

6.2 Transfer Control Block

This block is used to control the passing of signals between the AHB Interface and the External Interface Block. It is comprised of the following two sub-blocks:

- Transfer State Machine - used to control the flow of transfers to be performed

- Delay Timer and External Wait Control block - used to control the operation of reads, writes and external waits when a delay is required and also to control the operation of externally waited transfers.

These sub-blocks are described in detail in the following sections.

6.2.1 Transfer State Machine

The transfers performed through the external bus to the memory device are controlled by the state machine in this block. Moving between idle, read, write and turnaround states, the current state is used by the External Interface Block to generate the external output control signals at the appropriate instances. Depending on the type of transaction requested by the AHB master and other control signals from the External Interface Block, the state machine makes transitions to appropriate states. This is shown in the state diagram of ***Figure 6.2-1***. The corresponding state transition table is shown in ***Table 6.2-1***.

Error! Objects cannot be created from editing field codes.

Figure 6.2-1: Transfer State Machine block state diagram

Trans No.	Present State	State Change condition	Next State	Actions taken
-	Any State	RESET	ST_TSM_IDLE	From any state move to ST_TSM_IDLE
-	ST_TSM_IDLE	(WaitToutErr = 1)	ST_TSM_IDLE	(NextState = ST_TSM_IDLE)
C1	ST_TSM_IDLE	((MemWrReq =1) or (WrReqStr = 1) or (WtdWrReq = 1)) and (BufByPassCo = 1) and (SMBUSGNT = 1) and (Wait1cyc = 1) and ((WEnCntEZ = 1) or (WaitEn = 1))	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (SMBUSREQ = 1), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (ExtWrEnCo = 1)
C2	ST_TSM_IDLE	((MemWrReq =1) or (WrReqStr = 1) or (WtdWrReq = 1)) and (BufByPassCo = 1) and (SMBUSGNT = 1) and (Wait1cyc = 1) and (WEnCntEZ = 0) and (WaitEn = 0)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (SMBUSREQ = 1), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (WEnCntLdCo = 1)
-	ST_TSM_IDLE	((MemWrReq =1) or (WrReqStr = 1) or (WtdWrReq = 1)) and (BufByPassCo = 1) and ((SMBUSGNT = 0) or (Wait1cyc = 0))	ST_TSM_IDLE	(NextState = ST_TSM_IDLE), (SMBUSREQ = 1), (WrReqStr = 1)
C3	ST_TSM_IDLE	((MemWrReq =1) or (WrReqStr = 1) or (WtdWrReq = 1))	ST_TSM_BUFWR	(NextState = ST_TSM_BUFWR), (SMBUSREQ = 1),

		and (BufByPassCo = 0) and (Wait1cyc = 1)		
-	ST_TSM_IDLE	((MemRdReq = 1) or (RdRemain = 1) or (WtdRdReq = 1) or (RdReqStr = 1)) and ((SMBUSGNT =0) or (Wait1cyc = 0))	ST_TSM_IDLE	(NextState = ST_TSM_IDLE), (SMBUSREQ = 1), (RdReqStr = 1)
C4	ST_TSM_IDLE	((MemRdReq = 1) or (RdRemain = 1) or ((MWCfgDone = 1) and (WtdRdReq = 1)) or (RdReqStr = 1)) and (SMBUSGNT = 1) and (Wait1cyc = 1) and ((OenCntEZ = 1) or (WaitEn = 1))	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (SMBUSREQ = 1), (RdReqStr = 1), (SMCSEnCo = 1), (RdXdatEnCo = 1), (XoutEnCo = 1), (RdCntLdCo =1), (RdRemain = 0)
C5	ST_TSM_IDLE	((MemRdReq = 1) or (RdRemain = 1) or ((MWCfgDone = 1) and (WtdRdReq = 1)) or (RdReqStr = 1)) and (SMBUSGNT = 1) and (Wait1cyc = 1) and (OEnCntEZ = 0) and (WaitEn = 0)	ST_TSM_OENCNT	(NextState = ST_TSM_OENCNT), (SMBUSREQ = 1), (RdReqStr = 1), (SMCSEn = 1), (RdXdatEnCo = 1), (OEnCntLdCo = 1), (RdCntLdCo =1), (RdRemain = 0)
C6	ST_TSM_OENCNT	(DelayEnd = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (XoutEnCo = 1)
C7	ST_TSM_MEMRD	(WaitToutErr = 1)	ST_TSM_TURNAROUND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo = 1), (SMBUSREQ = 0), (SMCsEn = 0), (XoutDisCo = 1), (XdatDisCo = 1)
-	ST_TSM_MEMRD	(AhbRdEn = 1) and (iBMRdTrans = 1) and (HTransRegCo = NONSEQ) and (MemRdReq = 1) and (BankCmpCo = 1) and (SMBUSGNT = 1) and ((OEnCntEZ = 1) or (WaitEn = 1))	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (RdCntLdCo = 1), (XoutEnCo = 1)
C8	ST_TSM_MEMRD	(AhbRdEn = 1) and (iBMRdTrans = 1) and (HTransRegCo	ST_TSM_OENCNT	(NextState = ST_TSM_OENCNT), (RdCntLdCo = 1), (MemRdInd =

		= NONSEQ) and (MemRdReq = 1) and (BankCmpCo = 1) and (SMBUSGNT = 1) and (OEnCntEZ = 0) and (WaitEn = 0)		1), (OEnCntLdCo = 1), (XoutDisCo = 1)
C9	ST_TSM_MEMRD	(AhbRdEn = 1) and (iBMRdTrans = 1) and ((((MemRdReq = 1) and (BankCmpCo = 0)) and (SMBUSGNT = 1)) or ((HTransRegCo = IDLE) or ((MemRdReq = 0) and (MemWrReq = 0))))	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNAROND), (TrArCntLdCo = 1), (SMCsEnCo = 0), (XoutDisCo = 1), (XdatDisCo = 1), (RdReqStr = 1)
C9a	ST_TSM_MEMRD	(AhbRdEn = 1) and (iBMRdTrans = 1) and (((MemWrReq = 1) and (SMBUSGNT = 1)) or ((HTransRegCo = IDLE) or ((MemRdReq = 0) and (MemWrReq = 0))))	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNAROND), (TrArCntLdCo = 1), (SMCsEn = 0), (XoutDisCo = 1), (XdatDisCo = 1), (WrReqStr = 1)
C9b	ST_TSM_MEMRD	((MemRdOverCo = 1) and (SMBUSGNT = 0) and (iBMRdTrans = 1) and (iFastRdOp = 0))	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNAROND), (TrArCntLdCo = 1), (BrstAddIncCo == 1)
-	ST_TSM_MEMRD	(iBMRdTrans = 1) and (SMBUSGNT = 1) and (((CntEnd = 1) and (MemRdOverCo = 0)) or ((CntEZEnd = 1) and (MemRdOverCo = 0)))	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (AddrIncCo = 1), (RdBMcntLdCo = 1)
-	ST_TSM_MEMRD	(iBMRdTrans = 1) and (SMBUSGNT = 1) and (MemRdOverCo = 1) and (FastRdOp = 0) and (BMlenEnd = 0)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (BrstAddrIncCo = 1), (RdBMcntLdCo = 1)
-	ST_TSM_MEMRD	(iBMRdTrans = 1)	ST_TSM_MEMRD	(NextState =

		and (SMBUSGNT = 1) and (MemRdOverCo = 1) and (FastRdOp = 0) and (BMlenEnd = 1)		ST_TSM_MEMRD), (BrstAddrIncCo = 1), (RdcntLdCo = 1)
C10	ST_TSM_MEMRD	(iBMRdTrans = 1) and (SMBUSGNT = 1) and (MemRdOverCo = 1) and (FastRdOp = 1)	ST_TSM_AHBRD	(NextState = ST_TSM_AHBRD), (XoutDisCo = 1)
-	ST_TSM_MEMRD	(((CntEnd = 1) and (MemRdOverCo = 0)) or ((CntEZEnd = 1) and (MemRdOverCo = 0))) and ((WaitEn = 1) or (OEnCntEZ = 1)) and (BM = 1) and (SMBUSGNT = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (RdBMCntLdCo =1), (AddrIncCo = 1)
-	ST_TSM_MEMRD	(((CntEnd = 1) and (MemRdOverCo = 0)) or ((CntEZEnd = 1) and (MemRdOverCo = 0))) and ((WaitEn = 1) or (OEnCntEZ = 1)) and (BM = 0) and (SMBUSGNT = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (RdCntLdCo =1), (AddrIncCo = 1)
C11	ST_TSM_MEMRD	(((CntEnd = 1) and (MemRdOverCo = 0)) or ((CntEZEnd = 1) and (MemRdOverCo = 0))) and (WaitEn = 1) and (OEnCntEZ = 1) and (SMBUSGNT = 1)	ST_TSM_OENCNT	(NextState = ST_TSM_OENCNT), (AddrIncCo = 1) , (RdCntLdCo = 1), (OEnCntLdCo = 1), (XoutDisCo = 1)
C12	ST_TSM_MEMRD	(((CntEnd = 1) and (MemRdOverCo = 0)) or ((CntEZEnd = 1) and (MemRdOverCo = 0))) and (SMBUSGNT = 0)	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo = 1), (SMBUSREQ = 0), (RdRemain = 1), (XoutDisCo = 1), (SMCsEnCo = 0), (XdatDisCo = 1)
C13	ST_TSM_MEMRD	(MemRdOverCo = 1)	ST_TSM_AHBRD	(NextState = ST_TSM_AHBRD), (XoutDisCo = 1), (SMCsEnCo = 0)

C14	ST_TSM_AHBRD	(MemRdOver = 1) and (SMBUSGNT = 0)	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo = 1), (SMBUSREQ = 0), (SMCsEnCo = 0), (XdatDisCo = 1)
C15	ST_TSM_AHBRD	(MemRdReq =1) and (BankCmpCo = 1) and ((OEnCntEZ = 1) or (WaitEn = 1)) and ((AhbRdOver = 1) or (AhbRdEn = 1)) and (SMBUSGNT = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (Wait1Cyc = 1), (RdCntLdCo = 1), (XoutEnCo = 1), (SMCsEnCo = 1)
C16	ST_TSM_AHBRD	(MemRdReq =1) and (BankCmpCo = 1) and ((AhbRdOver = 1) or (AhbRdEn = 1)) and (SMBUSGNT = 1) and (BMlenEnd = 0) and (HTransRegCo = SEQ) and (BM = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (RdBMCntLdCo = 1), (XoutEnCo = 1), (SMCsEnCo = 1)
C17	ST_TSM_AHBRD	(MemRdReq =1) and (BankCmpCo = 1) and ((AhbRdOver = 1) or (AhbRdEn = 1)) and (SMBUSGNT = 1) and (OEnCntEZ = 0) and (WaitEn = 0) and ((BMlenEnd = 1) or (HTransRegCo /= SEQ) or (BM = 0))	ST_TSM_OENCNT	(NextState = ST_TSM_OENCNT), (RdCntLdCo = 1), (OEnCntLdCo = 1), (XoutDisCo = 1), (SMCsEnCo = 1)
C18	ST_TSM_AHBRD	(MemRdReq =1) and (BankCmpCo = 0) and ((AhbRdOver = 1) or (AhbRdEn = 1)) and (SMBUSGNT = 1)	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo =1), (XdatDisCo = 1), (XoutDisCo = 1), (SMCsEnCo = 0), (RdReqStr = 1)
C18a	ST_TSM_AHBRD	(MemWrReq =1) and ((AhbRdOver = 1) or (AhbRdEn = 1)) and (SMBUSGNT = 1)	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo =1), (XdatDisCo = 1), (XoutDisCo = 1), (SMCsEnCo = 0), (WrReqStr = 1)
-	ST_TSM_AHBRD	(Wait1cyc = 0)	ST_TSM_AHBRD	(NextState = ST_TSM_AHBRD), (Wait1cyc = 1)
C19	ST_TSM_AHBRD	(Wait1cyc = 1) and ((MemRdReq =0) and (MemWrReq	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (TrArCntLdCo =1), (XdatDisCo =

		=0))		1), (XoutDisCo = 1), (SMCsEnCo = 0), (Wait1cyc = 0)
C20	ST_TSM_TURNAROUND	((MemWrReq = 1) or (WrReqStr = 1) or (WtdWrReq = 1)) and ((CntEnd = 1) or (CntEZEnd = 1)) and (BufByPassCo = 1) and (SMBUSGNT = 1) and ((WEnCntEZ = 1) or (WaitEn = 1))	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (BufWrOvStrT = 1), (ExtWrEnCo = 1)
C21	ST_TSM_TURNAROUND	((MemWrReq = 1) or (WrReqStr = 1) or (WtdWrReq = 1)) and ((CntEnd = 1) or (CntEZEnd = 1)) and (BufByPassCo = 1) and (SMBUSGNT = 1) and (WEnCntEZ = 0) and (WaitEn = 0)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (BufWrOvStrT = 1), (WEnCntLdCo = 1)
C22	ST_TSM_TURNAROUND	((MemWrReq = 1) or (WrReqStr = 1) or (WtdWrReq = 1)) and ((CntEnd = 1) or (CntEZEnd = 1)) and (BufByPassCo = 0)	ST_TSM_BUFWR	(NextState = ST_TSM_BUFWR), (WrReqStr = 0)
C23	ST_TSM_TURNAROUND	((MemRdReq =1) or (WtdRdReq =1) or (RdReqStr = 1)) and ((WaitEn = 1) or (OEnCntEZ = 1)) and ((CntEnd = 1) or (CntEZEnd = 1)) and (SMBUSGNT = 1)	ST_TSM_MEMRD	(NextState = ST_TSM_MEMRD), (RdCntLdCo =1), (SMCSEnCo = 1), (XoutEnCo = 1), (RdXdatEnCo = 1)
C24	ST_TSM_TURNAROUND	((MemRdReq =1) or (WtdRdReq =1) or (RdReqStr = 1)) and (WaitEn = 0) and (OEnCntEZ = 0) and ((CntEnd = 1) or (CntEZEnd = 1)) and (SMBUSGNT = 1)	ST_TSM_OENCNT	(NextState = ST_TSM_OENCNT), (RdCntLdCo =1), (SMCSEnCo = 1), (OEnCntLdCo = 1), (RdXdatEnCo = 1)
C25	ST_TSM_TURNAROUND	(MemWrReq = 0) and (MemRdReq = 0) and (WtdWrReq = 0) and (WtdRdReq = 0) and (RdReqStr = 0) and (WrReqStr = 0) and ((CntEnd = 1) or (CntEZEnd = 1))	ST_TSM_IDLE	(NextState = ST_TSM_IDLE), (SMCsEnCo = 1), (XdatDisCo = 1), (XoutDisCo = 1), (SMBUSREQ =0)

-	ST_TSM_BUFWR	(SMBUSGNT = 0) and (BusDeGnt = 0)	ST_TSM_BUFWR	(NextState = ST_TSM_BUFWR), (SMBUSREQ = 0), (BusDeGnt = 1), (SMCsEn = 0)
		(BufWrOver = 1)		(BufWrOvStrT = 1)
-	ST_TSM_BUFWR	(SMBUSGNT = 0) and (BusDeGnt = 1)	ST_TSM_BUFWR	(NextState = ST_TSM_BUFWR), (SMBUSREQ = 1)
		(BufWrOver = 1)		(BufWrOvStrT = 1)
C26	ST_TSM_BUFWR	((BufWrOver =1) or (BufWrOvStrT = 1)) and (SMBUSGNT =1) and ((WEnCntEZ = 1) or (WaitEn =1))	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (SMCSEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (ExtWrEnCo =1), (BufWrOvStrT = 0), (BusDeGnt = 0)
C27	ST_TSM_BUFWR	((BufWrOver =1) or (BufWrOvStrT = 1)) and (SMBUSGNT =1) and (WEnCntEZ = 0) and (WaitEn = 0)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (SMCSEnCo = 1), (WrXdatEno = 1), (WrCntLdCo = 1), (WEnCntLdCo = 1), (BufWrOvStrT = 0), (BusDeGnt = 0)
C28	ST_TSM_WENCNT	(DelayEnd =1)	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (ExtWrEnCo =1)
-	ST_TSM_MEMWR	(Wait1cyc = 0) and (WaitToutErr = 1)	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (ExtWrDisCo = 1), (Wait1cyc = 1)
C29	ST_TSM_MEMWR	(Wait1cyc = 1) and ((WaitToutErr = 1) or (WaitToutErrD1 = 1))	ST_TSM_IDLE	(NextState = ST_TSM_IDLE), (SMBUSREQ = 0), (XdatDisCo = 1), (SMCsEnCo = 0), (Wait1cyc = 0)
C30	ST_TSM_MEMWR	((CntEnd =1) or (CntEZEnd = 1))	ST_TSM_SEQWR	(NextState = ST_TSM_SEQWR), (ExtWrDisCo = 0)
-	ST_TSM_SEQWR	(MemWrOver = 0) and (SMBUSGNT = 0) and (BusDeGnt = 0)	ST_TSM_SEQWR	(NextState = ST_TSM_SEQWR), (BusDeGnt = 1), (SMBUSREQ = 0), (SMCsEnCo = 0), (XdatDisCo = 1)
-	ST_TSM_SEQWR	(MemWrOver = 0) and (SMBUSGNT = 0) and (SMBUSREQ = 0) and (BusDeGnt = 1)	ST_TSM_SEQWR	(NextState = ST_TSM_SEQWR), (SMBUSREQ = 1)
C31	ST_TSM_SEQWR	(MemWrOver =0) and ((WEnCntEZ = 1) or (WaitEn =1))	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (AddrIncCo = 1), (SMCsEnCo = 1),

		and (SMBUSGNT = 1)		(ExtWrEnCo = 1), (WrCntLdCo =1), (WrXdatEnCo = 1), (BusDeGnt = 0)
C32	ST_TSM_SEQWR	(MemWrOver =0) and (WEnCntEZ = 0) and (WaitEn =0) and (SMBUSGNT = 1)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (AddrIncCo = 1), (SMCsEnCo = 1), (WEnCntLdCo =1), (WrCntLdCo =1), (WrXdatEnCo = 1), (BusDeGnt = 0)
C33	ST_TSM_SEQWR	(MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 1) and (BufByPassCo = 1) and (not(RBLE = 0 and RbleD1 = 1)) and ((WEnCntEZ = 1) or (WaitEn =1)) and (BnkAddStrQ = BnkAddStrCo)	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (ExtWrEnCo = 1)
C33a	ST_TSM_SEQWR	(Wait1cyc =1) and ((WEnCntEZ = 1) or (WaitEn =1))	ST_TSM_MEMWR	(NextState = ST_TSM_MEMWR), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (ExtWrEnCo = 1) (Wait1cyc = 0)
C34	ST_TSM_SEQWR	(MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 1) and (BufByPassCo = 1) and (not(RBLE = 0 and RbleD1 = 1)) and (WEnCntEZ = 0) and (WaitEn =0) and (BnkAddStrQ = BnkAddStrCo)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (WEnCntLdCo = 1)
C34a	ST_TSM_SEQWR	(Wait1cyc = 1) and (WEnCntEZ = 0) and (WaitEn =0)	ST_TSM_WENCNT	(NextState = ST_TSM_WENCNT), (SMCsEnCo = 1), (WrXdatEnCo = 1), (WrCntLdCo = 1), (WrReqStr = 0), (WEnCntLdCo = 1), (Wait1cyc = 0)
-	ST_TSM_SEQWR	(MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 1) and (BufByPassCo = 1) and (not(RBLE = 0 and RbleD1 = 1)) and (WEnCntEZ =	ST_TSM_SEQWR	(NextState = ST_TSM_SEQWR), (NextWait1cyc = 1)

		0) and (WaitEn =0) and (BnkAddStrQ != BnkAddStrCo)		
C35	ST_TSM_SEQWR	(MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 1) and (BufByPassCo = 0) and (RBLE = 0 and RbleD1 = 1)	ST_TSM_IDLE	(NextState = ST_TSM_IDLE) (SMCsEnCo = 0), (ExtWrDisCo = 0), (WrXdatEnCo = 0), (WrReqStr = 1)
C35a	ST_TSM_SEQWR	(MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 1) and (BufByPassCo = 0)	ST_TSM_BUFWR	(NextState = ST_TSM_BUFWR), (SMCsEnCo = 0)
C36	ST_TSM_SEQWR	(MemWrOver = 1) and (WtRdReq =1)	ST_TSM_TURNAROND	(NextState = ST_TSM_TURNARND), (ZeroIdleCo =1), (SMCsEnCo = 0), (XdatDisCo = 1)
C37	ST_TSM_SEQWR	(BusDeGnt = 0) or ((MemWrOver = 1) and (WtdWrReq = 1) and (SMBUSGNT = 0))	ST_TSM_IDLE	(NextState = ST_TSM_IDLE), (SMBUSREQ =0), (SMCsEnCo = 0), (ExtWrDisCo = 0), (XdatDisCo = 1)

Table 6.2-1 : Transfer State transition conditions and subsequent operations

Explanation of some basic types of transactions with reference to state transition table:

1. Example of read transfer sequence flow – Assume the Transfer State Machine (TSM) to be in the IDLE state.

- When **MemRdReq** or **WtdRdReq** signal is asserted by the AHB interface block or **RdReqStr** or **RdRemain** is asserted for a read transaction, then the TSM checks for the assertion **SMBUSGNT** signal after asserting **SMBUSREQ.** If it is de-asserted then the **SMBUSREQ** is kept asserted and **RdReqStr** is asserted.

- The TSM will remain in IDLE state till the bus is granted and once granted it asserts **RdCntLdCo**, **SMCsEnCo**, **RdXdatEnCo** and de-asserts **RdReqStr** and **RdRemain** then check if the **WSTOEN** value is greater than ‘0’ and whether the **WaitEn** (external wait control mode) bit is asserted. If **WSTOEN** is greater than zero and **WaitEn** is inactive, then the TSM moves to the OENCNT state, along with the assertion of the, **OEnCntLdCo**.

- Otherwise it moves to the MEMRD state and asserts **XoutEnCo**. If the TSM is currently in the OENCNT state, then it waits for the assertion of the **DelayEnd** signal from the Timer block. Once **DelayEnd** is asserted, then it asserts the signal **XoutEnCo** and moves to the MEMRD state.

- In the MEMRD state, if it is a Burst mode read then reads are done speculatively i.e. even before **HADDR** arrives the **SMCADDR** is generated. So it checks for whether the next transfer is a NONSEQ read for the same bank, if so stops the speculative read and starts a fresh read along with the assertion of **RdCntLdCo**. Here again if **WSTOEN** is greater than zero and **WaitEn** is inactive, then the TSM moves to the OENCNT state, along with the assertion of the **OEnCntLdCo**, **XoutDisCo**. Otherwise it moves to the MEMRD state and asserts **XoutEnCo**.

- On the other hand, if the next transfer is a read for different bank or a write transfer or a Idle transfer or no transfer is performed then the TSM moves to TURNARND state along with assertion of **TrArCntLdCo,**

XdatDisCo, XoutDisCo, RdReqStr (only for read for different bank), **WrReqStr**(only for write) and deassertion of **SMCsEnCo**.

- In the MEMRD state, if it is a Burst mode read and **SMBUSGNT** is removed and **MemRdOverCo** is high and **iFastRdOp** is not set state machine moves to TURNARND asserting **BrstAddIncCo**, **TrArCntLdCo**, **XoutDisCo**,**XdatDisCo** signals and deasserting **NextSMCsEn**.

- For a non Burst reads **CntEnd** or **CntEZEnd** assertion indicates that the access time is completed and the TSM polls for the assertion of the **MemRdOver** signal, which indicates that the read transaction has successfully completed. The TSM then moves to the AHBRD state. In case of either Burst or non Burst reads when HSIZE > MSize and **CntEnd** or **CntEZEnd** is asserted and **MemRdOver** is not asserted the TSM stays in MEMRD state and continues reading rest of the data from the memory. In case of HSIZE < MSize, once after the first read, the subsequent reads are given from the internal buffer in zero cycles saving the memory access time, so once the first read is over the TSM moves to AHBRD state.

- From the AHBRD state the TSM changes its next state depending upon the subsequent transaction request. If the **SMBUSGNT** is de-asserted, then the TSM asserts **SMBUSREQ** and moves to IDLE state. If it is a sequential read request to the same bank, then the TSM moves to either OENCNT state or the MEMRD state depending on the **OEnCnt** value or **WaitEn**. But if the subsequent transaction is a write or a read to a different bank, then the TSM moves to the TURNARND state and the **TrArCntLd** signal is asserted. If there are no transfers requested on the AHB bus then the TSM moves to the IDLE state and de-asserts **SMBUSREQ**, **SMCsEnCo** and asserts **XoutDisCo** signals. The **SMDATAEN** signal lines are activated for re-circulation.

2. Example of write transfer sequence flow - Assume the Transfer State Machine (TSM) to be in the IDLE state.

- When **MemWrReq** or **WtdWrReq** signal is asserted by the AHB interface block or **WrReqStr** is asserted for a write transaction, then the TSM asserts the **SMBUSREQ** signal. It also checks for **BufByPassCo** signal and **SMBUSGNT** signal to be asserted. If last two conditions are true, the TSM moves to either WENCNT state or the MEMWR state depending on the **WEnCnt** value or **WaitEn**. But if **SMBUSGNT** is de-asserted, it asserts **WrReqStr** and stays in IDLE state. The other possibility is that the **BufByPassCo** is de-asserted, in which case the TSM moves to the BUFWR state.

- In the BUFWR state the TSM checks if the **SMBUSGNT** is de-asserted and if true then it asserts the **BusDeGnt** and the **SMBUSREQ**. In parallel it checks for the assertion of **BufWrOver** and if so stores it as **BufWrOvStr** till Bus is granted again. Once the **SMBUSGNT** is asserted, it asserts **WrCntLdCo, WrXdatEnCo**, **SMCsEnCo** and de-asserts **WrReqStr**. Then the TSM checks if the **WSTWEN** value is greater than zero and whether the **WaitEn** (external wait control mode) bit is asserted. If the value is greater than zero and if **WaitEn** is inactive, then the TSM moves to the WENCNT state along with the assertion of the **WenCntLdCo**.

- Otherwise it moves to the MEMWR state and asserts **ExtWrEnCo**. If the TSM is currently in the WENCNT state, then it waits for the assertion of the **DelayEnd** signal from the Timer block. Once **DelayEnd** is asserted, then it asserts the **ExtWrEnCo** signal and moves to the MEMWR state.

- In the MEMWR state, if **WaitToutErr** is asserted then the TSM moves to IDLE state else the **CntEnd** or **CntEZEnd** assertion indicates that the access time is completed and the **ExtWrEnCo** signal is de-asserted and the TSM moves to the SEQWR state.

- From the SEQWR state, in cases where HSIZE > MSize after the first write **MemWrOver** would not be asserted so the TSM checks for **SMBUSGNT** assertion. If it is found asserted, the TSM checks whether the next address is to a different bank or not. If the next bank is a different bank the TSM waits for one more cycle and moves to MEMWR or WENCNT state depending on the **WenCnt** value or **WaitEn**. If access is to the same bank, TSM moves to MEMWR or WENCNT state depending on the **WenCnt** value or **WaitEn**. If **SMBUSGNT** is de-asserted it asserts **BusDeGnt** signal and then asserts the **SMBUSREQ** staying in the same state [i.e. SEQWR]. In other cases where **MemWrOver** is asserted TSM changes its next state depending upon the subsequent transaction request. If it is a write request, then the TSM checks for the assertion of **BufByPassCo**, if found asserted it moves to either WENCNT state or the MEMWR state depending on the **WEnCnt** value or **WaitEn**. But if **BufByPassCo** is de-asserted the TSM moves to BUFWR state. On the other hand if the subsequent transaction is a read, then the TSM moves to the TURNARND

state and the **ZeroIdleCo** signal is asserted. This ensures a single cycle turnaround. If there are no transfers requested on the AHB bus then the TSM moves to the IDLE state and de-asserts **SMBUSREQ, SMCsEnCo** signals and asserts **ExtWrDisCo**.

During external memory writes if the **HSIZE** is not same as memory width [MW] of the external memory device being accessed, then the single word buffer called **RdWrBuf** in the External Interface Block is used to build up the full data value. This allows transfers with **HSIZE** greater than **MW** to be performed with a single AHB transfer and multiple external transfers. When **MW** is greater than **HSIZE**, it allows sequential aligned AHB writes to load the buffer and then a single external write to be performed. When **HSIZE** is equal to **MW**, the write transfers are directly transferred to the external data bus register by bypassing the **RdWrBuf**. For these conditions, a signal called **BufBypassCo** is generated.

This **RdWrBuf** buffer is used in a similar manner for read transfers, allowing multiple sequential small AHB reads to be generated from a single large external read, and for large AHB reads to be generated from multiple smaller external reads.

The state machine is used to generate the read, write, turnaround, WEn delay and OEn delay timer/counter load signals to the Timer and Wait Control block - **RdCntLdCo**, **WrCntLdCo, TnCntLdCo, ZeroIdleCo, WEnCntLdCo** and **OEnCntLdCo** respectively, which are used to load the timer counter with the delay value required. The access/delay time counter block supplies the timeout signals **CntEnd** or **CntEZEnd** - for the access completion and **DelayEnd** – for the delay completion. These signals indicate when the timer/counter has reached zero and the next state may be entered. The **CntEZEnd** signal is specially generated when the counter values WST1, WST2, IDCY are zero. For all other cases, the **CntEnd** signal is used to indicate access time completion.

During an externally waited transfer, the double synchronised external wait input **SmWaitS2** is used to control the timing of the transfer instead of the counter expiry condition in the Timer and Wait Control block. The **CntEnd** signal to indicate that the transfer has completed successfully due to proper de-assertion of **SmWaitS2** input. The **WaitToutErr** output is used to indicate that the transfer did not complete due to a timeout on the **SmWaitS2** which is indicated by the assertion of the **CnclSmWaitS2** input line. Whenever this error condition is encountered, the TSM moves to the IDLE state.

Also the TSM is used to generate the **ExtWrEnCo, ExtWrDisCo, XoutEnCo, XoutDisCo, XdatDisCo, RdXdatEnCo, WrXdatEnCo** signals for External Interface block which is used to generate **nSMWEN, nSMBLS, nSMOEN, and nSMDATAEN** control signals.

The external memory bus turnaround cycles that are inserted for different write to read combinations and vice versa are implemented as indicated in the ***Table 6.2-2***.

Transaction		**No. of Turnaround cycles in terms of HCLK**
Write to Write	Same Bank	Zero cycles
Write to Write	Different Bank	Zero cycles
Write to Read	Same Bank	1 x HCLK
Write to Read	Different Bank	1 x HCLK
Read to Read	Same Bank	Zero cycles
Read to Read	Different Bank	IDCY x HCLK
Read to Write	Same Bank	IDCY x HCLK
Read to Write	Different Bank	IDCY x HCLK

Table 6.2-2 : Number of Bus Turn Around cycles inserted for different combinations

Note When there is an external wait timeout error (in the external wait enabled mode) during the course of a read transfer, bus turnaround must be performed. This is because SMWAIT could be de-asserted anytime and if the next access is a write or a read to a different, bank then there is a chance of bus contention.

The SmcCore uses the Data Bus Interface (DBI) to access the External Memory Data lines. When there is a valid read or a write transaction, the **SMBUSREQ** signal is asserted, indicating to the DBI that use of the bus is required. The DBI uses the **SMBUSGNT** line to indicate when there is a successful grant of the Bus to the SMC.

The interface to the Transfer State Machine block is shown in ***Figure 6.2-2***.

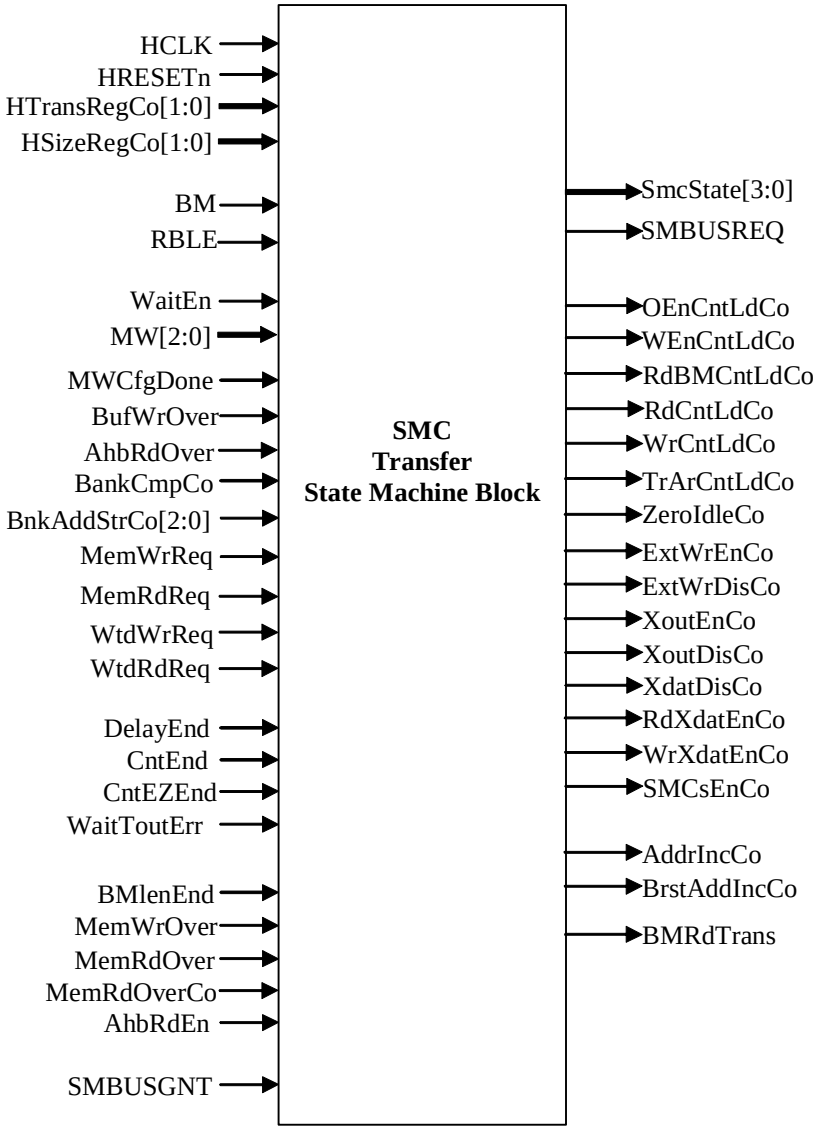

Figure 6.2-2 Transfer State Machine block

Signal Name	Type	Source/Destination	Description
HCLK	Input	Clock Source	System clock
HRESETn	Input	Reset Controller	System reset
HTransRegCo[1:0]	Input	AHB Interface block	Registered transfer type for the active transaction
HSizeRegCo[1:0]	Input	AHB Interface block	Registered transfer size active transaction
BM	Input	AHB Interface block	Burst ROM Device indicator
MW[1:0]	Input	AHB Interface block	Memory device width of the currently targeted bank
RBLE	Input	AHB Interface block	RBLE indicator
WaitEn	Input	AHB Interface block	External wait enable
MemRdReq	Input	AHB Interface block	Memory read initiation
MemWrReq	Input	AHB Interface block	Memory write initiation
WtdRdReq	Input	AHB Interface block	Stored read transfer request
WtdWrReq	Input	AHB Interface block	Stored write transfer request
BankCmpCo	Input	AHB Interface block	Indicates the successive transfers are to the same bank
BnkAddStrCo[2:0]	Input	AHB Interface block	Indicates the bank address
BufWrOver	Input	AHB Interface block	Indicates the completion of Write operation in Internal buffer
AhbRdOver	Input	AHB Interface block	Indicates the Completion of read by AHB
MWCfgDone	Input	AHB Interface block	Indicates the completion of MW bits programming after reset
CntEnd	Input	Timer & Wait control block	Timer counter expiry signal
CntEZEnd	Input	Timer & Wait control block	Timer counter expiry signal when count values are zero
DelayEnd	Input	Timer & Wait control block	WEn/OEn Count has ended
WaitToutErr	Input	Timer & Wait control block	Indicates that external wait input has reached timeout condition
MemWrOver	Input	External Interface block	Memory device Write completion signal
MemRdOver	Input	External Interface block	Memory device Read completion signal
MemRdOverCo	Input	External Interface block	Combinational version of MemRdOver

AhbRdEn	Input	External Interface block	Enabling signal to route data to HRDATA bus on read completion
BMlenEnd	Input	External Interface block	Signal to indicate Quad-Boundary address cross over
SMBUSGNT	Input	DBI	External bus grant signal from DBI
SmcState	Output	External Interface block	Current transfer state
SMBUSREQ	Output	DBI	External bus request
OEnCntLdCo	Output	Timer & Wait control block	Load output enable delay
WEnCntLdCo	Output	Timer & Wait control block	Load write enable delay
RdCntLdCo	Output	Timer & Wait control block	Load normal read access count
RdBMCntLdCo	Output	Timer & Wait control block	Load faster burst read access count
WrCntLdCo	Output	Timer & Wait control block	Load write access count
TrArCntLdCo	Output	Timer & Wait control block	Load turnaround delay
ZeroIdleCo	Output	Timer & Wait control block	1 cycle turnaround delay
ExtWrEnCo	Output	External Interface block	Enable for generation of **SMWEN** and **SMBLS** signals
ExtWrDisCo	Output	External Interface block	Disable for generation of **SMWEN** and **SMBLS** signals
XoutEnCo	Output	External Interface block	Enable signal for the **SMOEN**
XoutDisCo	Output	External Interface block	Disable signal for the **SMOEN**
XdatDisCo	Output	External Interface block	Disable signal for the **SMDATAEN**
RdXdatEnCo	Output	External Interface block	Signal to assert the proper byte lanes of external data bus depending on memory width during reads using the **SMDATAEN**
WrXdatEnCo	Output	External Interface block	Signal to assert all the byte lanes of external data bus during a write transfer using the **SMDATAEN**
AddrIncCo	Output	External Interface block	Signal for incrementing the Memory address **SMADDR**
BrstAddIncCo	Output	External Interface block	Signal for generating the Memory address **SMADDR** speculatively in advance in case for burst reads

BMRdTrans	Output	External Interface block	Signal indicates that currently a burst read transfer is in progress.
SMCsEnCo	Output	External Interface block	Chip select enable/disable signal

Table 6.2-3 Transfer State Machine block

6.2.2 Delay Timer and Wait Control

The 5-bit access time for external transfers and the 4-bit delay values for the WEN and OEN are controlled by the timer counter in this block. Control signals from the TSM block are used to load the timer with the relevant delay values. The **CntEnd** output is used to signal the end of access time for non zero access count values and **CntEZEnd** output is used to signal the end of access time when the relevant access values are zero. The **DelayEnd** output is used indicate the expiry of the delay value in case of non-zero chip select to WEN or OEN delay value.

Externally waited transfers also require the timer to be used to wait for the assertion of the **SMWAIT** input. But the same load enable signals are used for this purpose.

The following access/delay values are loaded by the timer:

- Read (single or start of burst) - WST1 (0 to 31)
- Read (during burst) - WST2 (0 to 31)
- Write - WST2 (0 to 31)
- Turnaround - IDCY (0 to 15)
- Write enable assertion delay
  - WSTWEN (0 to 15) (if WSTWEN value is programmed less than or equal to WST2)
  - WST2 (0 to 15) (if WSTWEN value is programmed greater than WST2)
- Read enable assertion delay
  - WSTOEN (0 to 15) (if WSTOEN value is programmed less than or equal to WST1 for non-burst mode devices)
  - WSTOEN (0 to 15) (if WSTOEN is programmed less than or equal to WST1 and WSTOEN less than or equal to WST2 for burst mode devices)
  - WST1 (0 to 15) (if WSTOEN is programmed greater than WST1)
  - WST2 (0 to 15) (if WSTOEN is programmed greater than WST2 for burst mode devices)
- External wait assertion - WST1 (0 to 31)

A simple state machine is used to control the operation of the Timer and Wait control. The state diagram of the Delay Timer and Wait Control block is shown in ***Figure 6.2-3***.

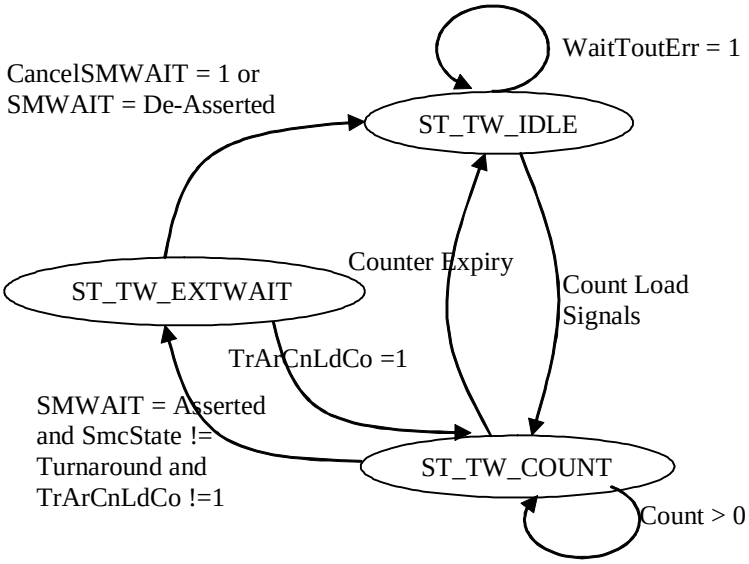

Figure 6.2-3: Delay Timer and Wait Control block state diagram

After reset, the state machine reaches IDLE state and **TimerCnt** is zero so the count end signal "CntEnd" will be asserted. Once if any of the turnaround, Output enable, write enable delay's load signals are asserted then the corresponding count values are loaded into the counter and the state machine enters into the count state ST_TW_COUNT. If external wait (**SmWaitS2**) is de-asserted and any of the write access, read access, burst read access count load values are asserted then the state machine moves into count state ST_TW_COUNT and loads the corresponding count value into the counter. It also de-asserts the count end, external wait error and wait timeout error signals. Check is performed to see whether the **SMWAIT** input due to previous transfer is already asserted irrespective of whether the targeted bank is **SMWAIT** controlled or not. This is because the **SMWAIT** could be de-asserted any time and there is then a chance of bus contention. There is no need to check for the WaitEn=1 because this is generic for both non-**SMWAIT** controlled and **SMWAIT** controlled.

The common counter which is loaded by any of the load enable signals is used for the following purposes:

1. To count down either the read access count or write access count or the turnaround cycle count. At the counter expiry, a common count completion signal 'CntEnd' signal is generated. This signal corresponds to the respective load enable signal.

2. In the external wait control mode, the counter is loaded with the count value which corresponds to the amount of time the SmcCore is expected to wait for the assertion of the **SMWAIT** input. If the **SMWAIT** is asserted within this stipulated time and TSM is not in turnaround or going to turnaround, then the state machine moves the EXT_WAIT state and the SmcCore is controlled by the **SMWAIT** input. On the other hand if the **SMWAIT** assertion is not detected within the counter expiry, then the transfer is completed successfully with no extra wait cycles.

During the Burst Read mode, when the subsequent reads are started in advance speculatively, then it is possible that the burst read is aborted. The access counting activity, which is already in progress will be aborted and depending on the next transfer {as indicated by the count load signals}, the counter is reloaded appropriately.

EXT_WAIT is the state in which the SmcCore is controlled by the **SMWAIT** input for the read or write access timings. De-assertion of the **SMWAIT**, signals the completion of the access time. It is also possible for some reason that the **SMWAIT** is not de-asserted by external controller, which means a potential hang situation. In such scenarios the **CANCELSMWAIT** input is asserted high, which indicates that a time-out has occurred on the de-assertion of the **SMWAIT**. When a time-out condition is encountered, the transfer is aborted, **WaitToutErr** flag is set and ERROR response is returned. ***Table 6.2-4***, shows the state transition conditions for the all the possible scenarios.

Present State	State Change Condition	Next State	Actions Performed
-	RESET	ST_TW_IDLE	
ST_TW_IDLE	(TrArCntLdCo =1) and (IdcyEZ = 0)	ST_TW_COUNT	(TimerCnt = IDCY)
ST_TW_IDLE	(WaitToutErr = 1) and (((WaitPolPrev = 0) and (SmWaitS2 = 0)) or ((WaitPolPrev = 1) and (SmWaitS2 = 1)))	ST_TW_IDLE	
ST_TW_IDLE	(WrCntLdCo = 1) and (WST2EZ = 0)	ST_TW_COUNT	(WaitToutErr = 0), (TimerCnt = WST2)
ST_TW_IDLE	(RdBmCntLdCo = 1) and (WST2EZ = 0)	ST_TW_COUNT	(WaitToutErr = 0), (TimerCnt = WST2)
ST_TW_IDLE	(RdCntLdCo = 1) and (WST1EZ = 0)	ST_TW_COUNT	(WaitToutErr = 0), (TimerCnt = WST1)
ST_TW_COUNT	(WaitEn = 1) and (SmWaitS2 = ASSERTED) and (SmcState != ST_TSM_TURNAROND) and (TrArnCntLdCo != 1)	ST_TW_EXTWAIT	(TimerCnt = 00000)
ST_TW_COUNT	(TrArCntLdCo =1) and (IdcyEZ = 0)	ST_TW_COUNT	(TimerCnt = IDCY)
ST_TW_COUNT	(RdBmCntLdCo = 1) and (WST2EZ = 0)	ST_TW_COUNT	(TimerCnt = WST2)
ST_TW_COUNT	(RdCntLdCo = 1) and (WST1EZ = 0)	ST_TW_COUNT	(TimerCnt = WST1)
ST_TW_COUNT	(WrCntLdCo = 1) and (WST2EZ = 0)	ST_TW_COUNT	(TimerCnt = WST2)
ST_TW_COUNT	(TimerCnt = 00001)	ST_TW_IDLE	(CntEnd = 1), (TimerCnt = 00000)
ST_TW_COUNT	(TimerCnt > 00001)	ST_TW_COUNT	(TimerCnt = TimerCnt – 1)
ST_TW_EXTWAIT	(TrArCntLdCo =1)	ST_TW_COUNT	(TimerCnt = IDCY)
ST_TW_EXTWAIT	(SmWaitS2 = DEASSERTED)	ST_TW_IDLE	(CntEnd = 1)
ST_TW_EXTWAIT	(CnclSmWaitS2 = ASSERTED)	ST_TW_IDLE	(WaitToutErr =1)

Table 6.2-4: Timer State transition conditions

This Figure shows the transition diagram for the WEN and OEN delay count and **DelayEnd** generation.

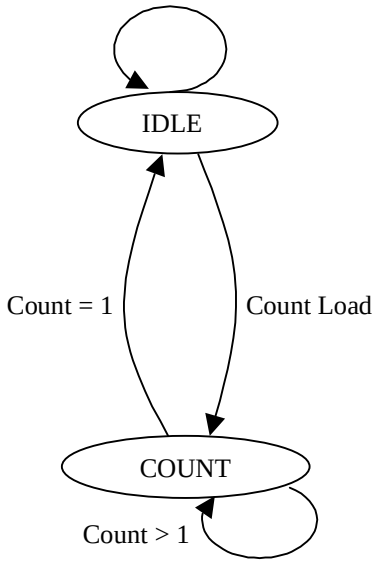

Figure 6.2.4: DelayEnd generation state diagram

The interface to the Delay Timer block is shown in ***Figure 6.2-5***.

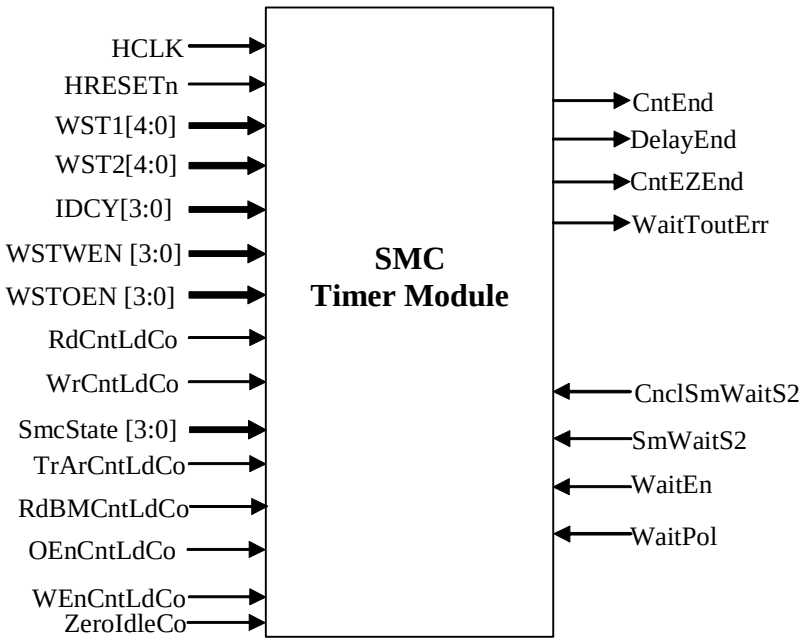

Figure 6.2-5 DelayTimer block

Signal Name	Type	Source/Destination	Description
HCLK	Input	Clock Source	System clock
HRESETn	Input	Reset Controller	System reset
OEnCntLdCo	Input	Transfer State Machine block	Load output enable delay
RdCntLdCo	Input	Transfer State Machine block	Load normal read access count value
RdBMLdCo	Input	Transfer State Machine block	Load burst read access count value
SmcState	Input	Transfer State Machine block	Smc state signal
WEnCntLdCo	Input	Transfer State Machine block	Load write enable access count value
WrCntLdCo	Input	Transfer State Machine block	Load write access count value
TrArCntLdCo	Input	Transfer State Machine block	Load turnaround delay
ZeroIdleCo	Input	Transfer State Machine block	1 cycle turnaround delay
WST1[4:0]	Input	AHB Interface block	Access count value
WST2[4:0]	Input	AHB Interface block	Access count value
IDCY[3:0]	Input	AHB Interface block	Turnaround count value
WSTWEN[3:0]	Input	AHB Interface block	Write enable delay value
WSTOEN[3:0]	Input	AHB Interface block	Output enable delay value
SmWaitS2	Input	Synchroniser block	Double Synchronised External wait input signal
CnclSmWaitS2	Input	Synchroniser block	Double Synchronised External wait termination input signal
WaitEn	Input	AHB Interface block	External wait enable signal for currently targeted bank.
WaitPol	Input	AHB Interface block	**SMWAIT** input polarity
WaitToutErr	Output	Transfer State Machine block	Indicates timeout error
DelayEnd	Output	Transfer State Machine block	Indicates delay count has ended
CntEZEnd	Output	Transfer State Machine block	Timer counter expiry signal when access/turnaround count values are zero
CntEnd	Output	Transfer State Machine block	Indicates access/turnaround count has ended

Table 6.2-5 Delay Timer block

6.3 Bus Interface Block

The Bus Interface block has three parts:

- The External Interface Block [EIB] – it provides the interface with the external world.

- Synchroniser Block – this module double synchronises the asynchronous inputs.
- Write Enable generation and Selection Block – this block is used to choose between the two sets of write enable signals

Each of the modules is explained in the following sub-sections.

6.3.1 External Interface Block [EIB]

This block provides the interface to the external bus. It contains the logic to generate the external control signals, and the 32-bit read/write buffer used to store the data value during transfers.

The state information passed from the Transfer State Machine in the Transfer Control block is used to determine what type of transfer is being currently performed, and to generate the external bus control signals during both read and write transfers. The operation of each transfer is described below.

There are three parts to a read transfer:

The first is setting the control output values used during the transfer, which are the address, chip select and output enable. The output enable delay (controlled by the WSTOEN bits of the memory bank control register) is used to delay the assertion of the output enable signal to save power depending on the operation of the external device. The delay is the number of cycles from the assertion of the chip select. The output enable is de-asserted at the same time as the chip select during all transfers.

The second is the delay for the read data to become valid, controlled by the WST1 bits of the memory bank control register. During this part all control signals remain constant until the end of the transfer, and at this time the read data value is expected to be stable. The memory data is sampled and at the same time the chip select and output enable lines are de-asserted. The read data value is stored in the 32-bit read/write buffer.

The last part of a read transfer is passing the data back to the AHB Interface block. The value stored in the read/write buffer is directly used to generate the AHB read data value on **HRDATA**.

During normal sequential read accesses to SRAM, flash and ROM, each external transfer is treated as a single access, so all control signals are de-asserted between each transfer and delays are applied to the assertion of the output enable line if required. As the properties of Burst ROM allow for sequential reads to be performed quicker than single reads, burst reads are performed differently. The initial access is performed as for any standard read transfer (with output enable delay if required), but subsequent reads are performed with the chip select and output enable lines held asserted and only the address changing, increasing the speed of the transfers. Also, the WST2 value is used to control the access time of the transfer, as it is expected that the data valid time of each burst transfer will be less than the data valid time of the initial transfer. At the end of the burst, the chip select and output enable lines are de-asserted together as for a standard read transfer. During burst reads the next transaction is started speculatively without seeing the AHB address **HADDR** value, so that performance is enhanced by saving clock. For this the TSM generates a signal called **BrstAddIncCo,** using which the EIB generates the next valid **SMCADDR** even before **HADDR** is sampled. The **SMCADDR** address generation logic takes into account the **HBURST** input and all the values of the **HBURST** are supported for a fixed burst read transfer. One exception when speculative reads are not performed is for reads with MSize > HSIZE. In this case, after the first read for the subsequent reads need not be accessed from memory instead can be read immediately with zero waits on the AHB by routing the data from the internal buffer, **RdWrBuf**. The subsequent sequential reads from memory will still be with the faster WST2 access value.

In all the above cases of burst read operation, the maximum length of the burst is limited by the quad-boundary crossover address. The effect of this is that, in the best case, when first address is aligned, one slow read and 3 fast reads will be performed from the Burst memory device. The quad-boundary address crossover is tracked by a signal called **BMlenEnd**. The quad-boundary crossover address depends on the memory width and for different memory width it is as follows:

- 8-bit device – SMCADDR[1:0] = “11”
- 16-bit device – SMCADDR[2:1] = “11”

- 32-bit device – SMCADDR[3:2] = “11”

There are three parts to a write transfer:

The first is loading the write data value into the read/write buffer. Due to the pipelined nature of AHB transfers, the write data is valid one cycle after the control signals, so during a read transfer the write data must be sampled at the correct time.

The second is similar to the first part of a read, setting the external control lines, which are the address, chip select, write data, byte lane select and write enable. The write enable delay (controlled by the WSTWEN bits of the memory bank control register) is used to delay the assertion of the write enable signal to save power depending on the operation of the external device. The delay is the number of cycles from the assertion of the chip select. The write enable is asserted on the falling edge of **HCLK** (rising edge of **nHCLK**) after the assertion of the chip select for zero wait states. The write enable will always be de-asserted half a cycle before the chip select, at the end of the transfer. The **nSMBLS** control lines will have the same timing as **nSMWEN** for writes to 8-bit devices [with **RBLE**=0] that use the byte lane selects as write enables, and the write enable outputs are unconnected. For 16 and 32-bit devices, the byte lane select signals have the same timing as the chip select [banks with **RBLE**=1].

Note The nSMWEN and nSMBLS signals are generated in the Write Enable Generation and Selection block.

The third part is the delay for the write operation to take place in the external device, controlled by the WST2 bits of the memory bank control register. During this part all control signals remain constant until the end of the transfer, when the chip select is de-asserted.

During sequential write accesses to SRAM and flash, the chip select lines are held asserted until the last write transfer, and the write enable is de-asserted between each transfer.

The operation of the interface is different during externally waited transfers controlled by the **SMWAIT** input (when the WaitEn bit of the control register has been set). As the timing of the transfer is unknown, the output/write enable is asserted immediately at the start of the transfer, and the WST1/2 value is not used to time the transfer, as de-assertion of **SMWAIT** is used to control this.

The behaviour of the SMC during writes to the memory device for different **HSIZE** to MSIZE relationships, are described below.

1. For AHB transfer with **HSIZE** of 32-bits :
    a. **MSIZE** of 8-bit: The 32-bit data is stored into the Internal Buffer, and four accesses are made to the memory to flush the 4 bytes from the Internal Buffer.
    b. **MSIZE** of 16-bits: The 32-bit data is stored into the Internal Buffer, and two accesses are made to the memory to flush the 2 half-words from the Internal Buffer.
    c. **MSIZE** of 32-bits: The Internal Buffer is by-passed and one access is made to the memory to flush the words from the external data output register.
2. For AHB transfer with **HSIZE** of 16-bits :
    a. **MSIZE** of 8-bits: The 16-bit data is stored into the Internal Buffer, and two accesses are made to the memory to flush the 2 bytes from the Internal Buffer.
    b. **MSIZE** of 16-bits: The Internal Buffer is by-passed and one access is made to the memory to flush the words from the external data output register.
    c. **MSIZE** of 32-bits : In the normal mode i.e. when the External Wait mode is disabled, if the first address is aligned then two half-words are stored into the Internal Buffer. Additionally, for this to happen, the second 16-bit write should be a SEQ transfer. So for two AHB write transfers HREADYOUT is given with zero waits. Finally one 32-bit access is made to the memory to flush the word from the Internal Buffer.

In the External Wait enabled mode or the normal mode, when the starting address is not aligned or when the first address is aligned but the second transfer is NONSEQ, then each half-word is individually stored into the Internal Buffer. Then each write to the memory is completed individually as a 16-bit access.

3. For AHB transfer with **HSIZE** of 8-bits :

   a. **MSIZE** of 8-bits: The Internal Buffer is by-passed and one access is made to the memory to flush the words from the external data output register.

   b. **MSIZE** of 16-bits : In the normal mode i.e. when the External Wait mode is disabled, if the first address is aligned then two bytes are stored into the Internal Buffer. Additionally, for this to happen, the second 8-bit write should be a SEQ transfer. One 16-bit access is made to the memory to flush the half-word from the Internal Buffer.

      In the External Wait enabled mode or the normal mode when the starting address is not aligned or when the first address is aligned but the second transfer is NONSEQ, then each byte is individually stored into the Internal Buffer. Then each write to the memory is completed individually as an 8-bit access.

   c. **MSIZE** of 32-bits : In the normal mode i.e. when the External Wait mode is disabled, if the first address is aligned then up to four bytes can be stored into the Internal Buffer. Additionally, for this to happen, the subsequent 8-bit writes should be a SEQ transfers. One 32-bit access is made to the memory to flush the word from the Internal Buffer, or if only two or three bytes are written to the buffer then 16-bit and 8-bit transfers are required to flush the buffer.

      In the External Wait enabled mode, the normal mode when the starting address is not aligned, or when the first address is aligned but the second transfer is NONSEQ, then each byte is individually stored into the Internal Buffer. Finally each write to the memory is completed individually as an 8-bit access.

In case where HSIZE < MSize, multiple AHB transfers are performed to fill the buffer. Till the buffer is filled, there will not be any external transfer. For this a signal called **BufWrOver** is routed from AHB Interface block which gets asserted once the buffer is filled.

In case where HSIZE > MSize, a single AHB transfer would fill the buffer and multiple external transfers will be performed in case of sequential writes. In such cases External interface block holds the TSM in MEMWR state using a signal called **MemWrOver**. This signal is asserted on completion of the flushes to the memory device or when the SMC is in MEMWR state and a WaitTOutErr happens.

The behaviour of the SMC during reads from the memory device for different **HSIZE** to MSize relationships are described below.

1. When **MSize** is 32-bits: In this case the **MemRdOver** signal is generated after a single 32-bit read access.

   a. **HSIZE** of 8-bit: A single word of data is read from the memory device and stored into the Internal Buffer. Then, depending on the sampled AHB transfer values, the relevant byte is routed out on the **HRDATA** bus. If sequential transfers are performed and if the first address is aligned, then only one 32-bit external read will be performed to the internal buffer. Read caching operation is supported as the subsequent three 8-bit AHB read data can be returned this buffer after the first byte is given out.

   b. **HSIZE** of 16-bits: A single word of data is read from the memory and stored into the Internal Buffer. Then, depending on the sampled AHB transfer values, the relevant half-word is routed out on the **HRDATA** bus. If sequential transfers are performed and if the first address is aligned, then only one 32-bit external read will be performed to the internal buffer. Read caching operation is supported as the subsequent 16-bit AHB read data can be returned this buffer after the first half-word is given out.

   c. **HSIZE** of 32-bits: The single word of data is read from the memory and stored into the Internal Buffer, and then routed out on the **HRDATA** bus.

2. When MSize is 16-bits : **HSIZE** of 8-bit: A single half-word of data is read from the memory device and stored into the Internal Buffer. Then, depending on the sampled AHB transfer values, the relevant byte is routed out on the **HRDATA** bus. If sequential transfers are performed, and the first address is aligned, then only one 16-bit external read will be performed to the internal buffer. Due to the read caching operation, the subsequent

byte can be returned from the **RdWrBuf** with zero wait cycles. The **MemRdOver** signal is generated after a single 16-bit read access.

a. **HSIZE** of 16-bits: The single half-word of data is read from the memory and stored into the Internal Buffer, and then routed out on the **HRDATA** bus. The **MemRdOver** signal is generated after a single 16-bit read access.

b. **HSIZE** of 32-bits: Two half-words of data are read from the memory device and stored into the Internal Buffer. Then, the word is routed out on the **HRDATA** bus. The **MemRdOver** signal is generated after a two 16-bit read accesses.

3. For **MSize** of 8-bits :

a. **HSIZE** of 8-bits: The single byte of data is read from the memory and stored into the Internal Buffer, and then routed out on the **HRDATA** bus. The **MemRdOver** signal is generated after a single 8-bit read access.

b. **HSIZE** of 16-bits: Two bytes of data are read from the memory device and stored into the Internal Buffer. Then, the half-word is routed out on the **HRDATA** bus. The **MemRdOver** signal is generated after two 8-bit read accesses.

c. **HSIZE** of 32-bits: Four bytes of data are read from the memory device and stored into the Internal Buffer. Then, the word is routed out on the **HRDATA** bus. The **MemRdOver** signal is generated after four 8-bit read accesses.

When multiple external transfers have to be performed in response to a single AHB transfer (i.e. AHB transfer size is larger than target external device width), a signal called **MemRdOver** is asserted when the last transfer has been completed on the external bus. This ensures that the Transfer State Machine starts the AHB side of the transfer only when the External Interface has finished all required transfers.

MemRdOver also gets asserted when there is a Wait time out error indicated by WaitTOutErr signal,.

When sequential address aligned byte or half-word AHB reads are performed from half-word or word external memory, it is possible that the read value required is already stored in t he read/write buffer, so an external access is not required. This is the read caching operation.

The types of memory devices connected determine how the write enable and byte lane control signals are used to generate byte, half-word and word transfers. The 8-bit devices must have the RBLE bit of the control register de-asserted. This ensures that during read transfers the byte lane select signals are driven HIGH, as these signals are used instead of the write enable signal. The 16-bit and 32-bit devices use both the write enable and byte lane select outputs to control write transfers, so should have RBLE asserted to ensure that during read transfers the byte lane strobes are driven LOW.

To reduce the number of external signal changes, the output data bus **SMDATAOUT** only changes value when a write transfer is performed i.e. its value is held constant with the previous write value at all other times. Unused bits of the data bus (during byte or half-word writes) will be driven with the data value from the previous write transfer. The data bus byte lanes controlled by the **nSMDATAEN[3:0]** data bus enables are shown in ***Table 6.3-1***.

nSMDATAEN[3:0]	SMDATAOUT
0	7:0
1	15:8
2	23:16
3	31:24

Table 6.3-1 : SMDATAOUT byte lanes controlled by nSMDATAEN lines

The block schematic of the External Interface Block is shown in ***Figure 6.3-1*****.**

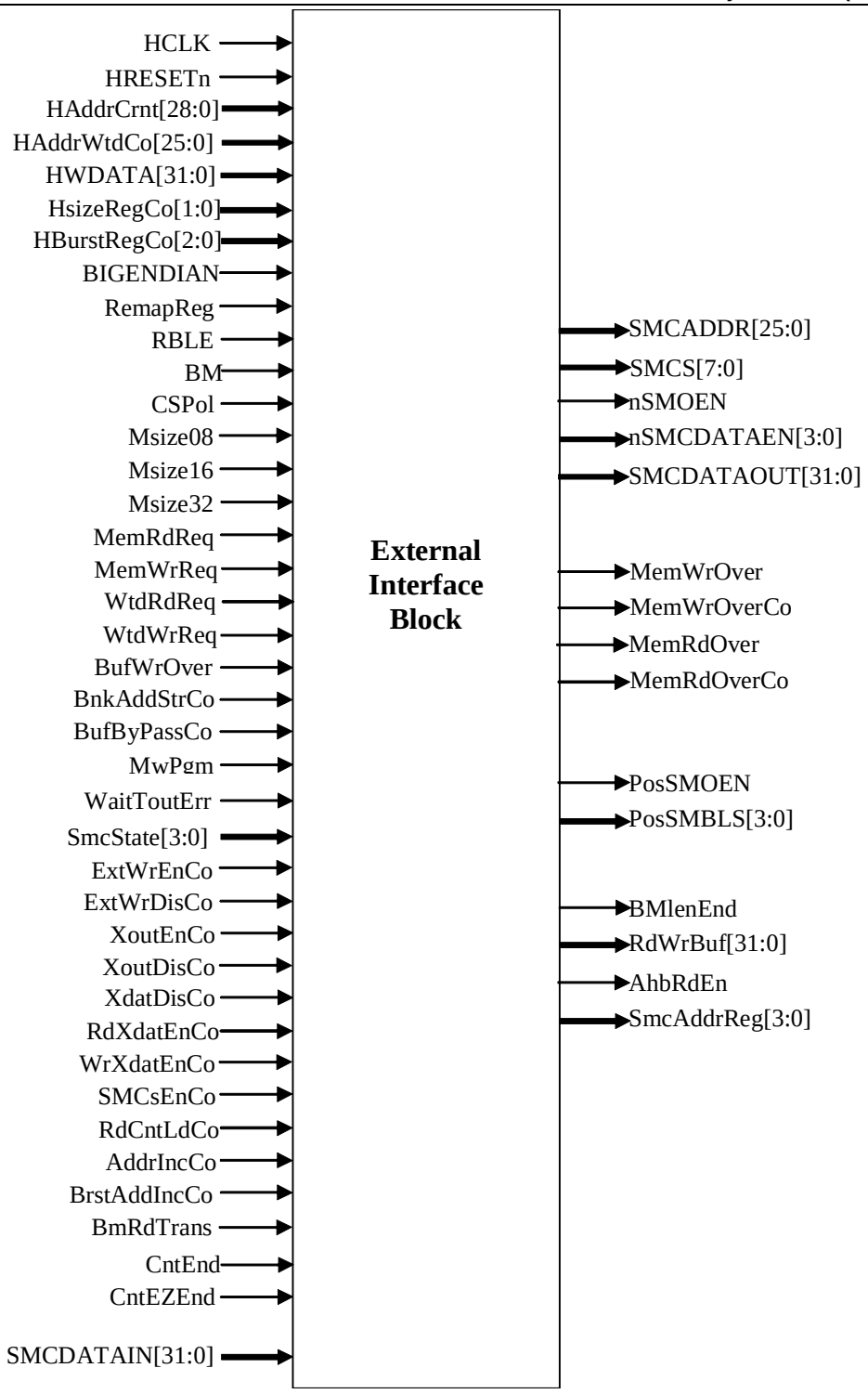

Figure 6.3-1 External Bus Interface Block

Signal Name	Type	Source/Destination	Description
HCLK	Input	Clock Source	System clock
HRESETn	Input	Reset Controller	System reset
HWDATA[31:0]	Input	AHB Interface block	System Data Bus
HSizeRegCo[1:0]	Input	AHB Interface block	Transfer size – Combinational
HBurstRegCo[2:0]	Input	AHB Interface block	Burst Size – Combinational
HAddrCrnt[28:0]	Input	AHB Interface block	Registered HADDR for a new transfer
HAddrWtdCo[25:0]	Input	AHB Interface block	The combinational version of HAddrCrnt for a waited memory transfer
BIGENDIAN	Input	AHB Interface block	Type of Endianness of the system
RemapReg	Input	AHB Interface block	Registered REMAP
RBLE	Input	AHB Interface block	Byte lane enabled device
BM	Input	AHB Interface block	Burst Memory
CSPol	Input	AHB Interface block	Chip select polarity
MSize08	Input	AHB Interface block	Signal to indicate that a 8-Bit memory is targeted
MSize16	Input	AHB Interface block	Signal to indicate that a 16-Bit memory is targeted
MSize32	Input	AHB Interface block	Signal to indicate that a 32-Bit memory is targeted
MemRdReq	Input	AHB Interface block	Memory read initiation
MemWrReq	Input	AHB Interface block	Memory write initiation
WtdRdReq	Input	AHB Interface block	Stored read transfer request
WtdWrReq	Input	AHB Interface block	Stored write transfer request
MwPgm	Input	AHB Interface block	Indicates that the Waited Read or Write Request is due to MW programming
BufWrOver	Input	AHB Interface block	Signal to flag completion of the write operation in the internal buffer
BnkAddStrCo	Input	AHB Interface block	Stored value of the Bank Address
BufByPassCo	Input	AHB Interface block	Indicates that the HSIZE = MSIZE and hence RdWrBuf can be bypassed
ExtWrEnCo	Input	Transfer State Machine block	Enable for generation of WE and BLS signals
ExtWrDisCo	Input	Transfer State Machine block	Disable for generation of WE and BLS signals
XoutEnCo	Input	Transfer State Machine block	Enable signal for the SMOEN
XoutDisCo	Input	Transfer State Machine block	Disable signal for the SMOEN

XdatDisCo	Input	Transfer State Machine block	Disable signal for the SMDATAEN
RdXdatEnCo	Input	Transfer State Machine block	Signal to assert the proper byte lanes of external data bus depending on memory width during reads
WrXdatEnCo	Input	Transfer State Machine block	Signal to assert all the byte lanes of external data bus during a write transfer and during Idle cycles
AddrIncCo	Input	Transfer State Machine block	Signal for incrementing the Memory address SMADDR
BrstAddIncCo	Input	Transfer State Machine block	Signal for incrementing the Memory address SMADDR in advance in case for burst reads
BMRdTrans	Input	Transfer State Machine block	Signal indicates that the current transfer is a burst read
SMCsEnCo	Input	Transfer State Machine block	Chip select enable
RdCntLdCo	Input	Transfer State Machine block	Load single read delay
CntEZEnd	Input	Timer & Wait control block	Timer counter expiry signal when count values are zero
CntEnd	Input	Timer & Wait control block	Indicates access/turnaround count has ended
WaitTOutErr	Input	Timer & Wait control block	Indicates that a Wait timeout error has occurred
SMDATAIN[31:0]	Input	External bus	External read data bus
SmcState	Input	Transfer Control block	Current transfer state
SMDATAOUT[31:0]	Output	External pads	External write data out
SMADDR[25:0]	Output	External pads	External address
SMCS[7:0]	Output	External pads	Chip select
nSMOEN	Output	External pads	Output enable
nSMDATAEN[3:0]	Output	External pads	Tri-state I/O pad enable for write data bus
MemWrOver	Output	AHB Interface block and Transfer State Machine block	Memory device Write completion signal
MemWrOverCo	Output	AHB Interface block and Transfer State Machine block	Combinational version of MemWrOver
MemRdOver	Output	AHB Interface block and Transfer State Machine block	Data Read completion from the Memory
MemRdOverCo	Output	AHB Interface block and Transfer State Machine block	Combinational version of MemRdOver
PosSMWEN	Output	Write Enable generation block	Positive edge (HCLK) triggered Write Enable, SMWEN
PosSMBLS[3:0]	Output	Write Enable generation block	Positive edge (HCLK) triggered byte lane select, SMBLS

BMlenEnd	Output	TSM & AHB Interface block	Burst length termination signal during Burst reads
RdWrBuf[31:0]	Output	Transfer State Machine block	Read data path from the EIB block to the HRDATA lines in the AHB interface block
AhbRdEn	Output	Transfer State Machine block	Enabling signal to route data to HRDATA bus on read completion
SmcAddrReg	Output	AHB Interface block	Registered version of SMCADDR

Table 6.3-2 External Interface Block

Table 6.3-2, gives the input/output signals of the External Interface Block

The chip select signals **SMCS** for the memory devices are generated in this block. The SMC supports programmable chip select polarity for each bank independently, using the **CSPol** bit of the control register. The **BnkAddrStrCo** from AHB interface block, which holds the bank number for the current transfer, is used to assert the chip select for the relevant memory bank during a transfer, depending on the polarity setting. All other chip selects are de-asserted depending on the polarity setting. The **RemapReg** input is also used to control the selection of memory banks zero and seven, as explained in Section 4.3 Memory Shadowing in the ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1]. The various combinations of SMCS for the different modes of **RemapReg** are shown in ***Tables 6.3-3*** and ***6.3-4***.

HADDR[28:26]	CSPol[x]	SMCS[x]	Memory configuration
000	0	0 (default)	Configuration for Bank 7
000	1	1	**SMCS[7]**
001	0	0 (default)	Configuration for Bank 1
001	1	1	**SMCS[1]**
010	0	0 (default)	Configuration for Bank 2
010	1	1	**SMCS[2]**
011	0	0 (default)	Configuration for Bank 3
011	1	1	**SMCS[3]**
100	0	0 (default)	Configuration for Bank 4
100	1	1	**SMCS[4]**
101	0	0 (default)	Configuration for Bank 5
101	1	1	**SMCS[5]**
110	0	0 (default)	Configuration for Bank 6
110	1	1	**SMCS[6]**
111	0	0 (default)	Configuration for Bank 7
111	1	1	**SMCS[7]**

Table 6.3-3 : Chip select table (When REMAP = 0). Bank0 will be shadowed by Bank7. Bank0 is not available and Bank7 will be at both SMCS[0] and SMCS[7]

HADDR[28:26]	CSPol[x]	SMCS[x]	Memory configuration
000	0	0 (default)	Configuration for Bank 0
000	1	1	**SMCS[0]**
001	0	0 (default)	Configuration for Bank 1
001	1	1	**SMCS[1]**
010	0	0 (default)	Configuration for Bank 2
010	1	1	**SMCS[2]**
011	0	0 (default)	Configuration for Bank 3
011	1	1	**SMCS[3]**
100	0	0 (default)	Configuration for Bank 4
100	1	1	**SMCS[4]**
101	0	0 (default)	Configuration for Bank 5
101	1	1	**SMCS[5]**
110	0	0 (default)	Configuration for Bank 6
110	1	1	**SMCS[6]**
111	0	0 (default)	Configuration for Bank 7
111	1	1	**SMCS[7]**

Table 6.3-4 : Chip select table (When REMAP = 1). All eight banks independently available.

6.3.2 Synchroniser Block

The asynchronous input signal **SMWAIT** and **CANCELSMWAIT** are passed through two registers so as to double synchronise them before using internally in the logic. The outputs of this block are the signals **SmWaitSync2** and **CnclSmWaitSync2.**

The block schematic of the Synchroniser Block is shown in the ***Figure 6.3-2***.

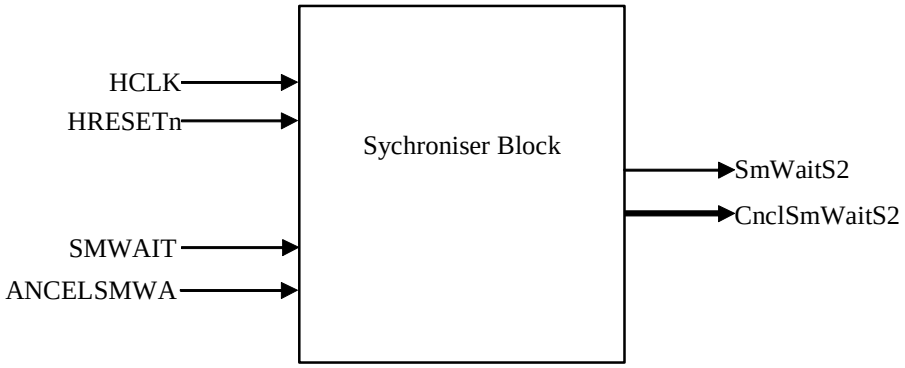

Figure 6.3-2 Synchroniser Block

6.3.3 Write Enable Generation and Selection Block

The SmcCore generates two sets of write enable signals for the **nSMWEN** & **nSMBLS[3:0]** outputs. One is a positive clock triggered set and the other is a negative clock triggered set. This block is used for the generation of the two sets of the write enable signals. The EIB provides the rising edge triggered write enables. These inputs are used to generate the falling edge triggered write enables signals. One of these two sets is selected by the control signal **PosWrEnSel** in this block. For the write transfers to the normal 8-bit wide devices, the negative clocked set for the **nSMBLS**/**nSMWEN** is selected. The positive clocked set for the **nSMBLS** is selected during write and read transactions for the RBLE enabled 16-bit or 32-bit devices, whereas the negative clocked set for the **nSMWEN** is selected during writes.

The block schematic of the Write Enable Generation and Selection Block is shown in the ***Figure 6.3-3***.

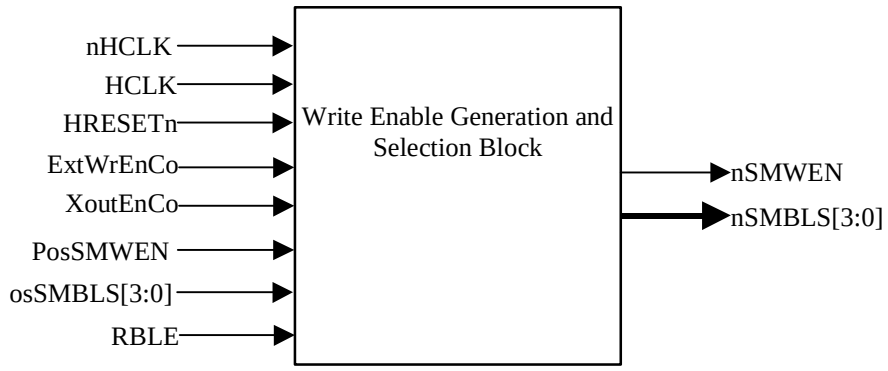

Figure 6.3-3 Write Enable Generation and Selection Block

7 OTHER MODULES IN THE SMC TOP LEVEL

7.1 Test Interface Controller (TIC) block

The TIC bus master module helps to allow externally applied test vectors to be converted into internal AHB transfers. The details of the TIC can be referred to, in the ***section 4.4, ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1]*** and ***chapter 6, of the AMBA Specification (Rev 2.0) [Ref. 3]***.

The block schematic of ***Figure 7.1-1***, shows the entity block of the TIC module and its I/O ports.

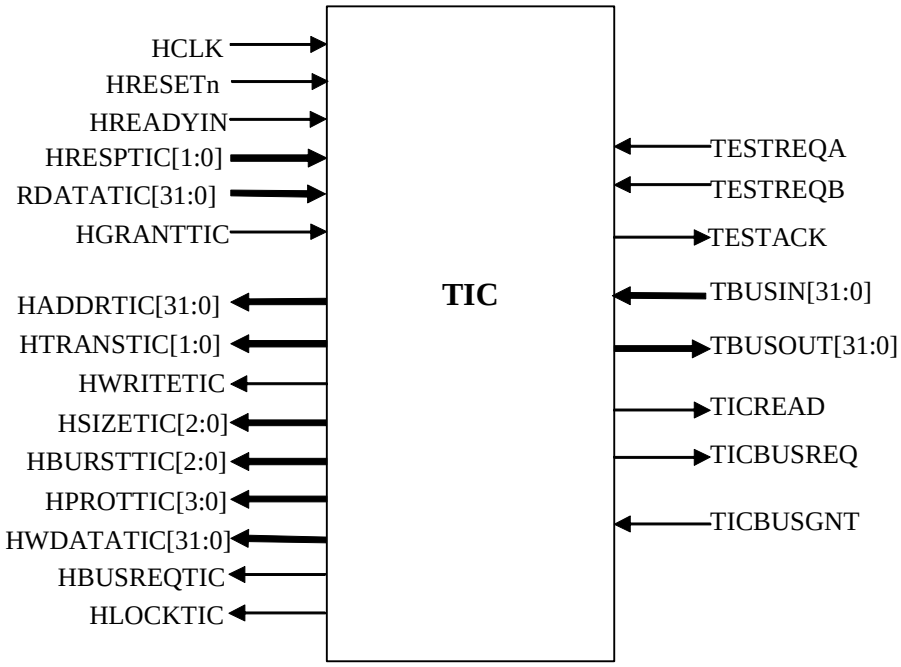

Figure 7.1-1 Test Interface Controller Block (TIC)

7.2 Data Bus Interface (DBI) block

This block implements dynamic priority arbitration logic for the External Data Bus and Address Bus Interface between SmcCore, TIC block and the Additional Memory Controller. The TIC has the highest priority followed by the Additional Memory Controller and the SmcCore has the lowest priority. If a lower priority master is doing transfers on the bus and a higher priority master requests for the bus then the DBI de-asserts the bus grant line of the lower priority master indicating it should leave control of the bus. Once the bus request line is de-asserted by the lower priority master, the DBI then grants the control of the bus to the higher priority master.

The Data Bus Interface implements the following functions:

- Multiplexes the Data lines and data bus byte lane enable lines from the TIC, Additional Memory Controller and the SmcCore on to the common Data pins of the chip.
- Arbitrates between the SmcCore and Additional Memory Controller for the control of the address lines.
- Implements the arbiter state machine to regulate access requests from the SmcCore block, TIC and Additional Memory Controller.

Support is also provided in the DBI so that the SmcCore can be interfaced with an external Bus interface arbiter block like EbiSdram. A bypass logic has been implemented which will cater to the requirement when the SMC needs to interface with the EbiSdram. In this scenario the DBI becomes redundant as the External BusReq and BusGnt pins become active and these get connected to the SmcCore block. During synthesis the redundant DBI can get optimised out. This choice can be made by using the **EXTBUSMUX** pin, which needs to be tied to the appropriate logic level and then synthesised to get the required functional feature. The details regarding the DBI can be found in ***section 4.5, ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref. 1].*** If the internal DBI is bypassed for using the external EbiSdram, then the bus-request and bus-grant pins of the SmcCore & TIC, along with TIC's output data lines and data enable pin [all these are pins of the SMC] will become active.

The block schematic shown in the ***Figure 7.2-1****,* represents the diagram of the DBI with its I/O pins.

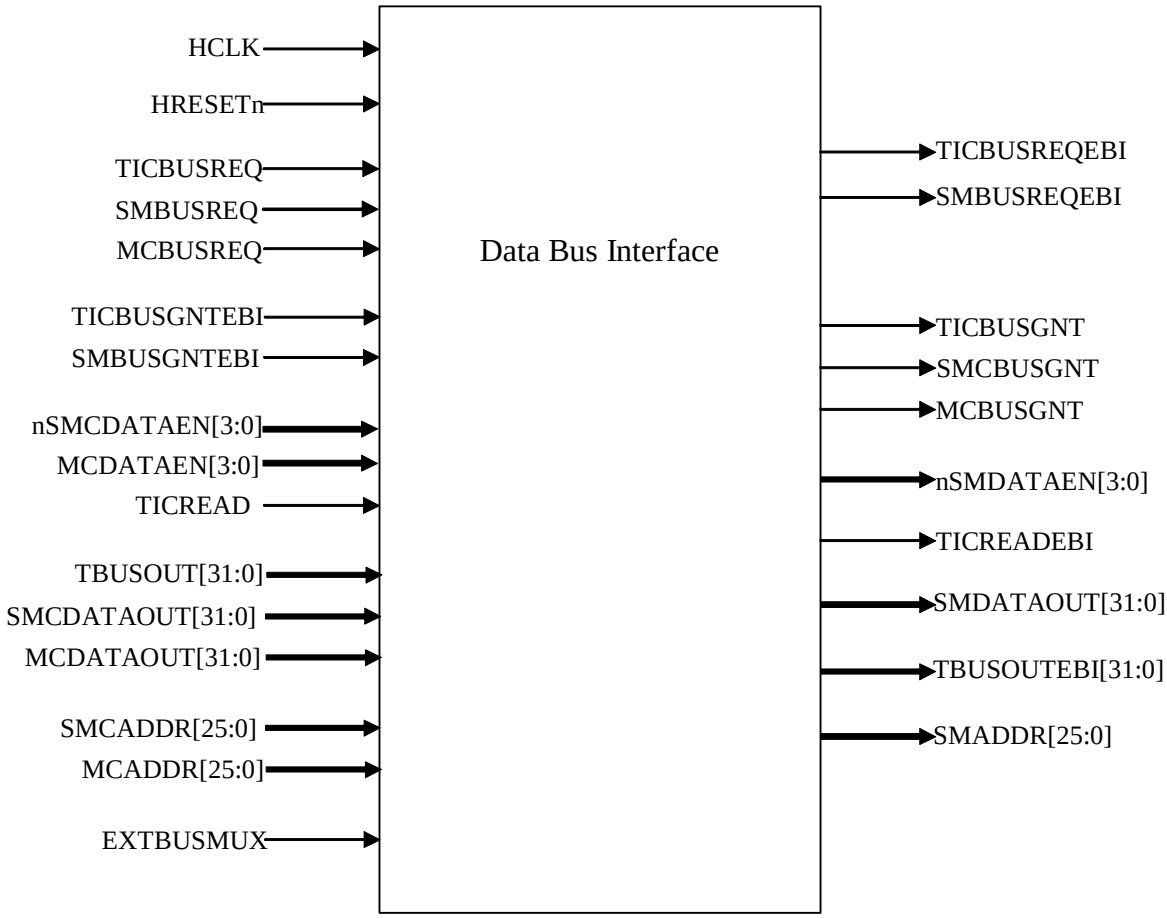

Figure 7.2-1 The Data Bus Interface [DBI] Block

7.3 Revision Designator block (SmcRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed. The port level connections for the module are as below.

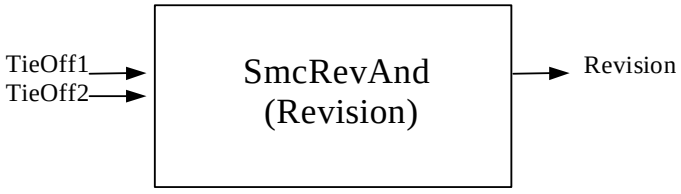

Figure 7.3-1 Block Inputs and Outputs for SmcRevAnd

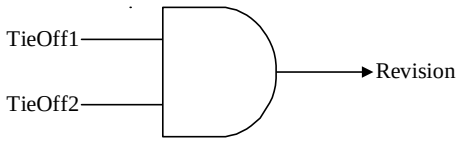

Figure 7.3-2 RevisionAnd component

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the SMC, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the AHB slave interface logic block, SmcAhbif, in the SmcCore block. This helps software to identify the revision number through an AHB read.

8 SOME SMC IMPLEMENTATION RELATED NOTES

Software Assumptions

1. Only 32-bit transfers should be performed to any of the internal registers of the SmcCore block. It is the responsibility of the master to ensure the same. It should be noted that transfers of any other size will still be carried out by using the appropriate lines of the **HWDATA** and **HRDATA** bus. No specific ERROR response will be given for a transfer size other than 32-bit.
2. It is assumed that once **REMAP** is set HIGH, it can only be set LOW again by a system reset. If the **REMAP** input changes to LOW in between, then it will cause **SMCS[0]** to be shadowed by **SMCS[7**].
3. In the External Wait enabled mode the software has to take care that proper values are loaded for the counter values, which take into account the delay seen by the SmcCore due to double synchronisation. Also the controller must ensure that the **SMWAIT** is asserted properly in relation to the count value programmed in the WST1/WST2 control register bits. In worst case the SMWAIT needs to be asserted at least 2 cycles before the counter expiry, as the SMC is controlled by the synchronised external wait signal.
4. The default reset value of the WSTWEN register for all the banks has been configured as “0001”. This will introduce one cycle delay by default between the SMCS to SMWEN/SMBLS. This has been introduced taking to into account that the EbiSdram could be used instead of the internal DBI. The EbiSdram function introduces one cycle delay on the final address and data lines before routing it to the pads. If the user decides to use the internal DBI for arbitration, then the WSTWEN values can be programmed to “0000” by the software so as to make use of the fastest possible performance.

9 SMC BLOCK VERIFICATION STRATEGY

Figure 9-1 illustrates the SMC Testbench.

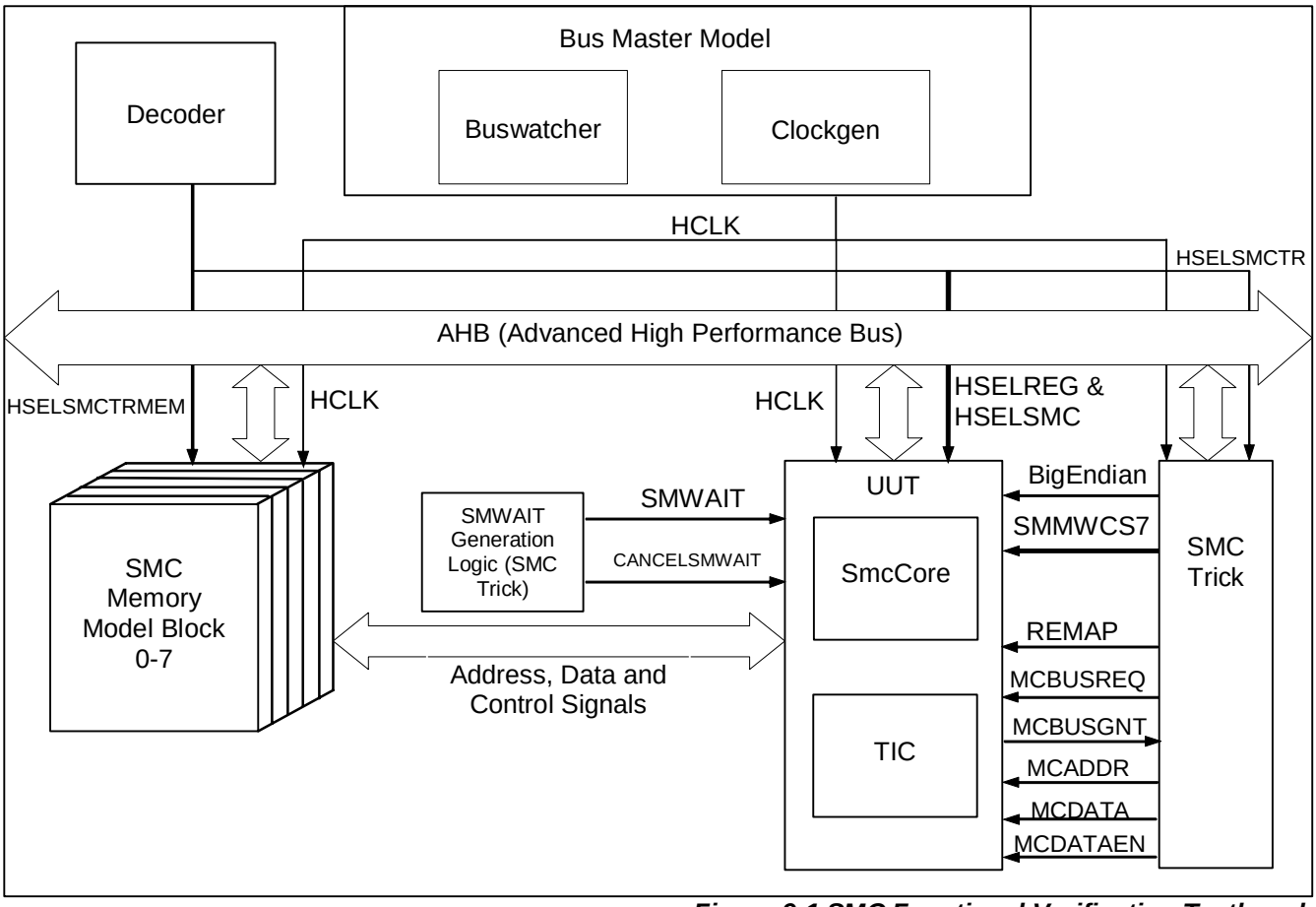

Figure 9-1 SMC Functional Verification Testbench

SMC Verification is done with the help of Trickbox which is a programmable AHB block designed to provide a means of stimulating the non-AMBA input pins of the SMC and capturing the non AMBA outputs. Both VHDL and Verilog versions of the Trickbox have been developed.

The Testbench contains eight instantiations of TrickMem block, which serves the purpose of emulating memory models. These will be used for the verification of the SMC. The Testbench also has a SMC Trickbox block, which is used to verify the non-memory related signals. It is possible to access the Memory Models (TrickMem), both from the SMC and the AHB. Additionally, Denali memory models have also been used to verify the SMC, so as to emulate a practical real life memory device behaviour in the testing environment. Details of these blocks are given in the following section.

9.1 SMC Trickbox

SMC Trickbox is used to capture non-AMBA, non-memory related signals. SMC Trickbox is used to drive SMMWCS7, REMAP, SMBUSGNT and BIGENDIAN inputs of SmcCore.

9.1.1 SMC Trickbox Signal Description

The following tables summarise the SMC Trickbox signal list. ***Table 9.1-1*** lists the AHB interface signals while ***Table 9.1-2*** lists the block specific non-AMBA signals.

Signal Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock. This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.
HRESETn	IN	Reset Controller	Bus reset (active low).
HADDR[4:2]	IN	AHB Master	The 3-bit system address bus.
HTRANS[1:0]	IN	AHB Master	Indicates the type of the current transfer.
HSIZE[2:0]	IN	AHB Master	Indicates the size of the current transfer.
HWRITE	IN	AHB Master	AHB transfer direction signal, indicates a write access when HIGH and read access when LOW.
HRDATA[31:0]	OUT	To Test Bench	AHB read data bus. It is used to transfer data from bus slaves to the master during read operations.
HWDATA[31:0]	IN	AHB Master	AHB write data bus. During write operations, HWDATA is used to transfer data from the bus master to the slaves.
HREADYIN	IN	External Slave(s)	The slave will start the transfer only if HREADYIN is HIGH.
HREADYOUT	OUT	AHB Slave	HREADYOUT along with an OK response indicates the successful completion of a data transfer.
HRESP[1:0]	OUT	AHB Slave	The bus transfer response.
HSELSMCTR	IN	AHB Decoder	SMC Trick select

Table 9.1-1: AMBA AHB signals description

Signal Name	Type	Source / Destination	Description
SMMWCS7[1:0]	OUT	SMC	To control static configuration bits to indicate the memory width used for boot memory bank seven 00 = 8-bit 01 = 16-bit 10 = 32-bit 11 = 8-bit
REMAP	OUT	SMC	Indicates the state of the memory map: 0 = reset memory map (SMCS[0] mapped to SMCS[7])

			1 = normal memory map. All the chip selects and the corresponding memory banks are available
ENDIANCNT	OUT	SMC	This static configuration bit indicates the type of Endian-ness of the system. 1 = BIG Endian mode 0 = LITTLE Endian mode
SMDATAOUT[31:0]	IN	SMC	External Data Out signal from the SMC. It has been taken in to check for 'X'es on the bus.
SMADDR[25:0]	IN	SMC	Memory address bus of the SMC. It has been taken in so as to check for 'X'es on the bus.
SMCS[7:0]	IN	SMC	Chip Select signals of each bank. It has been taken in so as to check for 'X'es on the bus.
nSMDATAEN[3:0]	IN	SMC	Data bus enable signal. It has been taken in so as to check for 'X'es on the bus.
nSMWEN	IN	SMC	Memory Write enable signal. It has been taken in so as to check for 'X'es on the line.
nSMBLS[3:0]	IN	SMC	Memory Bus Lane Enable signals. It has been taken in so as to check for 'X'es on the bus.
SMCActLowCS[7:0]	IN	SMCTrickMem	Active low Chip Select signals from SMCTrickMem. These are used to route the SMCTrWAIT signal to the SMWAIT line of the SMC appropriately.
SMWAIT	OUT	SMC	External Wait input signal into the SMC.
nSMWAIT	OUT	To TestBench	External Wait input signal into the Test Bench.
nSMOEN	IN	SMC	Memory output enable signal. It has been taken in so as to check for 'X'es on the line.
CANCELSMWAIT	OUT	SMC	External WAIT Cancel Signal. This signal is asserted when the SMWAIT signal gets timed out. The time out period is set as 32 cycles of HCLK.
MCBUSREQ	OUT	SMC	Bus access request signal from the Additional Memory Controller (AMC). Assertion of this signal indicates that the additional memory controller has requested for the external bus.
MCBUSGNT	IN	SMC	Bus access grant signal to the AMC. This signal indicates that the additional memory controller has been granted the control of the external bus.
SMBUSREQ	IN	SMC	This is the SMBUSREQ from SMC when EXTBUSMUX =1. It is connected to SMBUSREQEBI of SMC.
SMBUSGNT	OUT	SMC	Grant signal generated by looking at SMBUSREQ when EXTBUSMUX = 1.
EXTBUSMUX	OUT	SMC	Indicates to SMC whether the external bus grant is given by another agent. If EXTBUSMUX is 0 SMC generates

			SMBUSGNT.
MCADDR[25:0]	OUT	SMC	Address bus of the AMC.
MCDATA[31:0]	OUT	SMC	Data bus of the AMC.
MCDATAEN	OUT	SMC	Data Enable signals of the AMC.

Table 9.1-2 Block specific non-AMBA Trickbox signals

9.1.2 SMC Trickbox Functional Description

The SMC Trickbox contains the following major modules.

The block diagram is shown in the ***Figure. 9.1-1.***

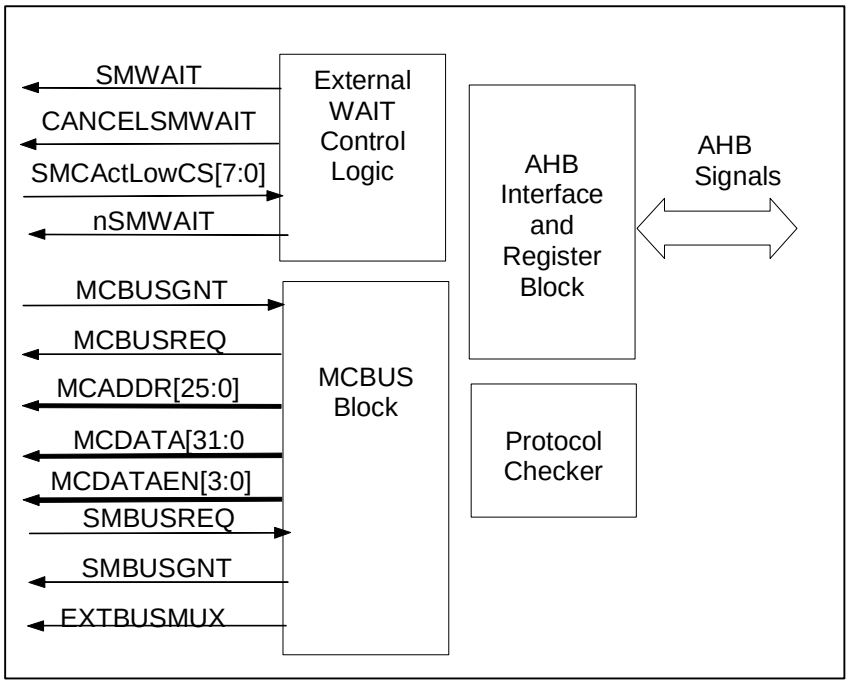

Figure 9.1-1 SMC Trickbox Block Diagram

AHB Interface and Register Block

This block interfaces the SMC Trickbox with the AHB. The Trickbox is an AHB slave and it supports read-write operations of transfer size 32-bits and burst type as INCR. The AHB interface block performs the following operations.

- Generation of all AHB slave related response.
- Generation of read and write decodes for all registers.

SMC Trickbox Register Summary

The SMC Trickbox registers are shown in ***Table 9.1-3***.

Address	Read Location	Write Location
SMC Trick Base + 0x0000	SMCTrMWCS	SMCTrMWCS

SMC Trick Base + 0x0004	SMCTrRemap	SMCTrRemap
SMC Trick Base + 0x0008	SMCTrEndian	SMCTrEndian
SMC Trick Base + 0x000C	SMCTrCS2WTR0	SMCTrCS2WTR0
SMC Trick Base + 0x0010	SMCTrCEWTR0	SMCTrCEWTR0
SMC Trick Base + 0x0014	SMCTrCS2WTR1	SMCTrCS2WTR1
SMC Trick Base + 0x0018	SMCTrCEWTR1	SMCTrCEWTR1
SMC Trick Base + 0x001C	SMCTrCS2WTR2	SMCTrCS2WTR2
SMC Trick Base + 0x0020	SMCTrCEWTR2	SMCTrCEWTR2
SMC Trick Base + 0x0024	SMCTrCS2WTR3	SMCTrCS2WTR3
SMC Trick Base + 0x0028	SMCTrCEWTR3	SMCTrCEWTR3
SMC Trick Base + 0x002C	SMCTrCS2WTR4	SMCTrCS2WTR4
SMC Trick Base + 0x0030	SMCTrCEWTR4	SMCTrCEWTR4
SMC Trick Base + 0x0034	SMCTrCS2WTR5	SMCTrCS2WTR5
SMC Trick Base + 0x0038	SMCTrCEWTR5	SMCTrCEWTR5
SMC Trick Base + 0x003C	SMCTrCS2WTR6	SMCTrCS2WTR6
SMC Trick Base + 0x0040	SMCTrCEWTR6	SMCTrCEWTR6
SMC Trick Base + 0x0044	SMCTrCS2WTR7	SMCTrCS2WTR7
SMC Trick Base + 0x0048	SMCTrCEWTR7	SMCTrCEWTR7
SMC Trick Base + 0x004C	SMCTrMCREQD	SMCTrMCREQD
SMC Trick Base + 0x0050	SMCTrGNT2RMREQ	SMCTrGNT2RMREQ
SMC Trick Base + 0x0054	SMCTrMCADDR	SMCTrMCADDR
SMC Trick Base + 0x0058	SMCTrMCBUSR	SMCTrMCBUSR
SMC Trick Base + 0x005C	SMCTrCNCLWAIT	SMCTrCNCLWAIT
SMC Trick Base + 0x0058	SMCTrEBICntl	SMCTrEBICntl

Table 9.1-3 SMC Trickbox Register map

Protocol Checker Block

This block checks the following protocols of the SMC.

- Checking for ‘X’es on the SMC output signals.
- Checking for simultaneous assertion of multiple Memory Chip Select bits.
- Checking if any of the control signals like SMWEN/SMBLS, SMOEN are asserted when the chip select for all the banks (SMCS[7:0]) are de-asserted.

Whenever a violation is found in the SMC protocol, the Protocol Checker module flags error messages.

External Wait Control Logic

This block does the following functions:

- Generates SMWAIT signal according to the polarity and the delays programmed.
- Asserts the CANCELSMWAIT signal when the SMWAIT signal gets timed out.
- The timeout period of SMWAIT signal cane be programmed.

MCBUS Block

This block does the following functions:

- Generates the MCBUSREQ signal with a programmable delay.
- De-asserts the MCBUSREQ signal after the MCBUSGNT has been received. The delay after which it is de-asserted is programmable.
- Generates the MCADDR, MCDATA and MCDATAEN signals to the SMC.
- Checks the SMADDR, SMDATA and nSMDATAEN lines. When a mismatch is found, warning messages are flagged.
- It generated EXTBUSMUX signal depending one the value programmed in SmcTrEbiCntl.
- If the EXTBUSMUX = 1 then it generates SMBUSGNT looking at SMBUSREQ from the SMC.

9.1.3 SMC Trickbox Register Description

The base address of the SMC Trickbox is not fixed and may be different based on system implementations. However, the offset of any particular register from the base address is fixed.

SMCTrMWCS: SMC Trickbox Memory Width Register

Bit	Name	R/W	Reset value	Function
1- 0	MWCS	R/W	Unaffected by the HRESETn	MWCS bits determine the width of the boot memory. These bits are clocked but are not affected by HRESETn so values are programmed in these bits after Reset is over and these values remain even after reset is re-asserted. These bits are used to check SMMWCS7 functionality. 0x00 for 8-bit wide memory 0x01 for 16-bit wide memory 0x10 for 32-bit wide memory 0x11 for 8-bit wide memory

Table 9.1-4 SMC Trickbox Memory width register

SMCTrRemap: SMC Trickbox Remap Register

Bit	Name	R/W	Reset value	Function

0	REMAP	R/W	Unaffected by the HRESETn	Indicates the state of the memory map: 0 = reset memory map In this mode SMCS[7] shadows SMCS[0] i.e. both SMCS[7] and SMCS[0] selects TrickMem 7 (bank7) and (SMCS[0] gets mapped to SMCS[7]) 1 = normal memory map This bit is clocked but is not affected by HRESETn so value is programmed in this bit after Reset is over and these values remain even after reset is re-asserted. This bit is used drive REMAP input of SMC.

Table 9.1-5 SMC Trickbox Remap register

SMCTrEndian: SMC Trickbox Endian Register

Bit	Name	R/W	Reset value	Function
0	Endian	R/W	0x00	This static configuration bit indicates the type of Endian-ness of the system. The value put in this register is driven on BIGENDIAN input of SMC. 0 = LITTLE Endian mode 1 = BIG Endian mode

Table 9.1-6 SMC Trickbox Endian register

SMCTrCS2WTR0-7: SMC Trickbox Chip Select to SMWAIT assertion delay Register

Bit	Name	R/W	Reset value	Function
25	WaitEn	R/W	0x0	This bit indicates whether the external wait timing is enabled for the currently targeted Bank. 0 = External Wait disabled. 1 = External Wait Enabled.
24	WaitPol	R/W	0x0	This bit indicates the polarity of the SMWAIT for currently targeted Bank. 0 = Active LOW SMWAIT 1 = Active HIGH SMWAIT
23-0	CS2WTRx	R/W	0x00	Chip Select assertion to SMWAIT assertion delay count for Bank x. Chip select assertion or an event on the SMADDR is taken as the trigger point and the SMWAIT is asserted after a delay of (CS2WTRx) x Tclk.

Table 9.1-7 SMC Trickbox Chip Select to SMWAIT assertion delay register

SMCTrCEWTR0-7: SMC Trickbox SMWAIT assertion to SMWAIT de-assertion delay Register

Bit	Name	R/W	Reset value	Function
23-0	CEWTRx	R/W	0x00	SMWAIT assertion to SMWAIT de-assertion delay count for Bank x. The SMWAIT is de-asserted after a delay of (CEWTRx) x Tclk.

Table 9.1-8 SMC Trickbox SMWAIT assertion to SMWAIT de-assertion delay register

SMCTrMCREQD: SMC Trickbox MCBUSREQ assertion delay Register

Bit	Name	R/W	Reset value	Function
4-0	MCREQD	R/W	0x00	MCBUSREQ assertion delay count. An access to the location SMCTrMCBUSR causes the MCBUSREQ signal to be asserted after a delay of (MCREQD) x Tclk.

Table 9.1-9 SMC Trickbox MCBUSREQ assertion delay register

SMCTrGNT2RMREQ: SMC Trickbox MCBUSGNT to MCBUSREQ de-assertion delay Register

Bit	Name	R/W	Reset value	Function
4-0	GNT2RMREQ	R/W	0x00	This is the delay count between the assertion of the MCBUSGNT and the de-assertion of the MCBUSREQ. The delay is (GNT2RMREQ) x Tclk.

Table 9.1-10 SMC Trickbox MCBUSGNT to MCBUSREQ de-assertion delay register

SMCTrMCADDR: SMC Trickbox MCADDR Register

Bit	Name	R/W	Reset value	Function
25-0	MCADDR	R/W	0x00	This programmable register value is driven out through the MCADDR lines to the SMC. When the MCBUS is granted, the SMC drives out this value through the SMADDR lines.

Table 9.1-11 SMC Trickbox MCADDR register

SMCTrMCBUSR: SMC Trickbox MCBUSR Register

Bit	Name	R/W	Reset value	Function

-	MCBUSR	R/W	-	An access (read/write) to this location triggers the MCBUSREQ generation logic. The MCBUSREQ will be asserted after a delay of (MCREQ) x Tclk. Writes to this address drive out the write data through the MCDATA lines to the SMC. Read does not affect the MCDATA lines. This is used to toggle the MCDATA bus output.

Table 9.1-12 SMC Trickbox MCBUSR register

SMCTrCNCLWAIT : SMC Trickbox CANCELWAIT Register

Bit	Name	R/W	Reset value	Function
5	CancelSMWAITEn	R/W	0x0	Enable CANCELSMWAIT functionality of trickbox.
4:0	CancelWaitCntr	R/W	0x1F	Indicates the number of clocks after which SMWAIT is active, CANCELSMWAIT becomes active.

Table 9.1-13 SMC Trickbox CANCELSMWAIT register

SMCTrEBICntl : SMC Trickbox EBI Control Register

Bit	Name	R/W	Reset value	Function
12	EXTBUSMUX	R/W	0x0	If EXTBUSMUX = 0 SMC generates the SMBUSGNT else MCBUS block in trickbox generated the SMBUSGNT.
9:5	DeGntCounter	R/W	0x0	Indicates the number of clocks after which SMBUSGNT is given, SMBUSGNT is pulled low.
4:0	GntCounter	R/W	0x0	Indicates the number of clocks after which SMBUSREQ is given, SMBUSGNT is made high.

Table 9.1-14 SMC Trickbox CANCELSMWAIT register

9.2 SmcCore Memory Model (TrickMem)

The TrickMem serves the purpose of a memory model, required for the verification of the SmcCore.

The following features of the TrickMem are programmable.

- Memory width – can be programmed as 8-bit, 16-bit and 32-bit
- Memory type – can be programmed as SRAM, ROM and BURST ROM
- Read access time
- Write access time

- Memory data bus turn around time
- Independent chip select polarity for all the 8 memory banks
- Polarity and Enabling of the External asynchronous wait input signal
- Delayed WEN assertion during writes to Memory devices with the programmable delay
- Delayed OEN assertion during Reads from Memory devices with the programmable delay
- Configurable size at reset for boot memory bank using external control pins

The following sections detail the functionality of the TrickMem.

9.2.1 TrickMem Signal Description

The following tables summarise the SmcCore TrickMem signal list. ***Table 9.2-1*** lists the AHB interface signals while ***Table 9.2-2*** lists the block specific non-AMBA signals.

Signal Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock. This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.
HRESETn	IN	Reset Controller	Bus reset (active low).
HADDR[15:0]	IN	AHB Master	Address bus. Subset of 32-bit system address bus.
HTRANS[1:0]	IN	AHB Master	Indicates the type of the current transfer.
HSIZE[2:0]	IN	AHB Master	Indicates the size of the current transfer.
HBURST[2:0]	IN	AHB Master	Indicates the type of burst value of transfer
HWRITE	IN	AHB Master	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW.
HRDATA[31:0]	OUT	To Test Bench	AHB read data bus. It is used to transfer data from bus slaves to the master during read operations.
HWDATA[31:0]	IN	AHB Master	AHB write data bus. During write operations, HWDATA is used to transfer data from the bus master to the slaves.
HREADYIN	IN	External Slave(s)	The slave will start the transfer only if HREADYIN is HIGH.
HREADYOUT	OUT	AHB Slave	HREADYOUT along with an OK response indicates the successful completion of a data transfer.
HRESP[1:0]	OUT	AHB Slave	The bus transfer response.
HSELSMCTRMEM	IN	AHB Decoder	SMC TrickMem select

Table 9.2-1: AMBA AHB signals description

Signal Name	Type	Source / Destination	Description

SMDATA[31:0]	INOUT	SMC	Memory data bus. It is a 32-bit bi-directional bus
SMADDR[25:0]	IN	SMC	Address bus to the memory bank.
nSMWEN	IN	SMC	Write enable for the external memory bank (active low).
nSMOEN	IN	SMC	Output enable for external memory bank (active low).
nSMBLS[3:0]	IN	SMC	Byte enables (byte-wide) for the external memory bank (active low).
SmcTrAllCS[7:0]	IN	SMC	The CS status of all the eight banks connected with the SMC.
CANCESMWAIT	IN	Trickbox	CANCELSMWAIT signal from trickbox.
nSMWAIT	IN	Trickbox	Indicates whether SMWAIT is active or not.
SmcTrCS	IN	Testbench top level	Individual Select line for a TrickMem (bank).
SmcActLowCS	OUT	Testbench top level	Used for checking the multiple assertion of Chip Select.

Table 9.2-2 Block specific non-AMBA Memory signals

9.2.2 TrickMem Functional Description

The TrickMem contains the following major modules.
The block diagram of the TrickMem is shown in the ***Figure 9.2-1***

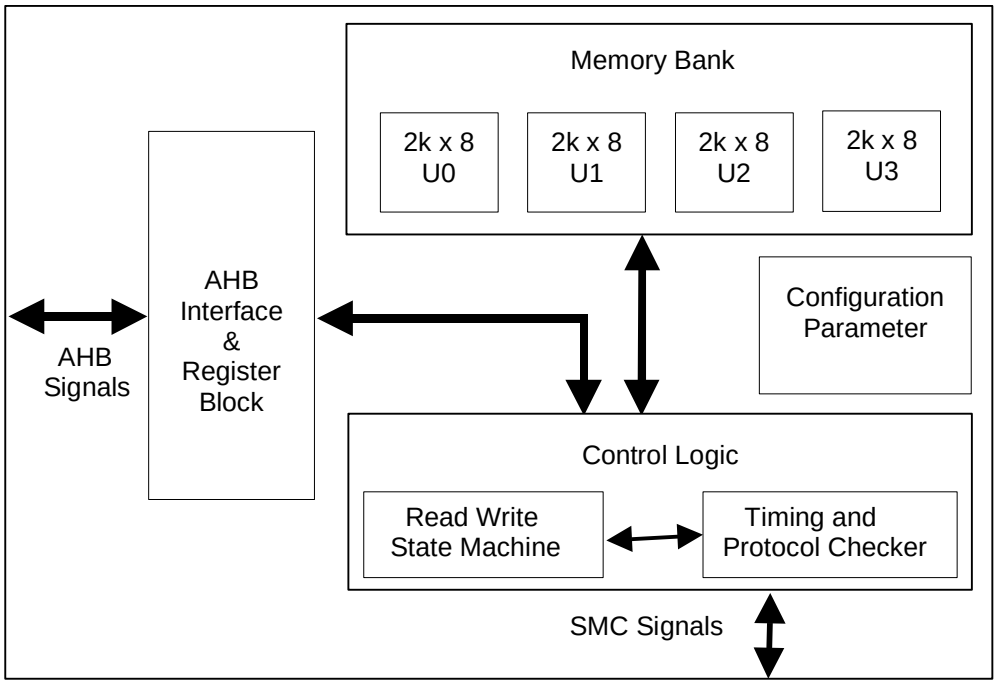

Figure 9.2-1 TrickMem block diagram

AHB Interface and the Register Block

This block interfaces the TrickMem with the AHB. The TrickMem is an AHB slave and it supports read/write operations of transfer size 32-bit and burst type as INCR. The AHB interface block performs the following operations.

- Generation of all AHB slave related response.
- Generation of read and write decodes for all registers.

TrickMem Register Summary

The SMC TrickMem registers are shown in ***Table 9.2-3***.

Address	Read Location	Write Location
SMC TrickMem Base + 0x0000 – 1FFF	SMCTrMEMR	SMCTrMEMR
SMC TrickMem Base + 0x2000	SMCTrIDCY	SMCTrIDCY
SMC TrickMem Base + 0x4000	SMCTrWST1	SMCTrWST1
SMC TrickMem Base + 0x6000	SMCTrWST2	SMCTrWST2
SMC TrickMem Base + 0x8000	SMCTrMEMT	SMCTrMEMT
SMC TrickMem Base + 0xA000	SMCTrMEMB	SMCTrMEMB
SMC TrickMem Base + 0xB000	SMCTrCS2OEN	SMCTrCS2OEN
SMC TrickMem Base + 0xB800	SMCTrCS2WEN	SMCTrCS2WEN
SMC TrickMem Base + 0xC000	SMCTrCSPOL	SMCTrCSPOL

Table 9.2-3 SMC Register map

The Memory model (TrickMem) is instantiated eight times for eight-memory banks of SMC. Different base addresses are used for different Memory models.

Memory Block

This block contains a basic memory array of 8-bit width and 2K depth. This memory array is instantiated four times in parallel to create 8Kb of memory of 32-bit width and 2K depth. Depending on the value programmed in the SMCTrMEMT register, it is possible to configure this block as an 8-bit, 16-bit or a 32-bit wide memory. Parameter value MemDeep, in the SMCTrConst file determines the depth of the memory block. It is possible to access this memory both from the AHB and from the SMC. It is ensured that data does not get written if CANCELSMWAIT comes during a memory write transfer.

Control Logic

The Control logic contains two parts:

1. State machine

The state machine is shown in ***Figure 9.2-2***. It contains the following states

ST_IDLE: This is the default state after reset. When the Write Enable, chip select is asserted, the state machine moves to the ST_WRITE state if the Memory type is SRAM. Similarly, the Output Enable, chip select is asserted, the state machine moves to the ST_READ state, if the Memory type is BURSTROM it moves to ST_INITBURSTREAD state. The state machine remains in this state if the access is out of range.

ST_WRITE: This is the state the state machine stays during memory write operation. The state machine can move only to the ST_IDLE state from this state. For this it checks the nSMWEN or nSMBLS deassertion.

ST_READ: This is the state the state machine stays during memory read operation for non burst cases. The state machine can move only to the ST_IDLE state from this state. For this it checks the nSMOEN de-assertion. Else if it is a continuous read the state machine stays in this state it self.

ST_INITBURSTREAD: This is the state the state machine stays during memory read operation for the initial read of a burst memory case. The state machine can move either to the ST_IDLE state or to the ST_BURSTREAD state. It checks the nCS de-assertion, if so moves to ST_IDLE state. Otherwise if nCS is asserted and nSMOEN is also asserted then it means subsequent burst reads are expected, so it moves to ST_BURSTREAD. In the case of nSMOEN getting deasserted with nCS asserted, the state machine remains in ST_INITBURSTREAD state. It remains in the ST_INITBURSTREAD state in case the address crosses a quad word boundary.

ST_BUSRTREAD: This is the state the state machine stays during memory read operation for the subsequent reads in case of burst memory. The state machine can move either to the ST_IDLE state or to the ST_INITBURSTREAD state. It checks the nCS de-assertion, if so moves to ST_IDLE state. It moves into ST_IDLE state when nSMOEN gets deasserted when HSIZE is less than MSIZE. Otherwise if the burst is terminated it moves ST_INITBURSTREAD state. In case the access is to a different page, it moves into ST_INITBURSTREAD else if the access is to the same page it remains in the same state.

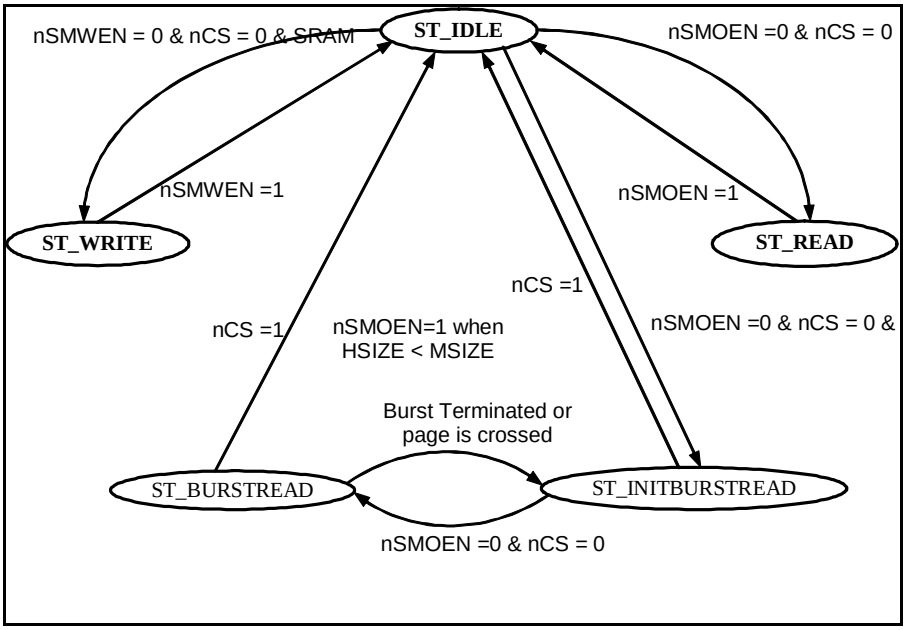

Figure 9.2-2 TrickMem state machine

1. Timing and protocol check block, which displays error messages when the following unexpected operations happen:

- Access time violation for non-burst read: The access time starts with assertion of chip select and ends with de-assertion of output enable or chip select or address change.
- Access time violation for burst read: The access time starts with change in Address and ends with change in address or de-assertion of chip select.

In the above two protocol checks, the actual access time measured is compared with (WST1/WST2 register).

- Access time violation for write: The access time starts with assertion of chip select and ends with de-assertion of write enable.
- Maximum time limit violation for non-burst read access : This maximum access time is little more than normal access time as mentioned in Protocol 1

- Maximum time limit violation for burst read access. This maximum access time is little more than normal access time as mentioned in Protocol 2
- Maximum time limit violation for write access : This maximum access time is little more than normal access time as mentioned in Protocol 3
- Read and write enable asserted at the same time
- Address changes when write enable is active
- Chip select controlled write
- Chip select changes when read is active
- Address hold time violation for write accesses with respect to write enable de-assertion
- Data hold time violation for write accesses with respect to write enable de-assertion
- Data set-up time violation for write accesses with respect to write enable de-assertion
- Write to read idle time violation for same and different bank with idle time as one clock period.
- Read to write idle time violation for same and different bank with idle time as (1 + IDCY) * clock period.
- Read to read idle time violation for different bank with idle time as (1 + IDCY) * clock period
- Write enable byte select violation
- Write to ROM or BURST ROM or Write Protected SRAM.
- Chip select assertion to Output Enable assertion time violation. This check is not performed for Wait enabled mode
- Chip select assertion to Write Enable assertion time violation. This check is not performed for Wait enabled mode
- Chip select changes when Write is active.
- nSMBLS protocol checks with respect to RBLE.
- nSMWEN/nSMBLS or the nSMOEN assertion checking if all the Chip Selects (SMCS[7:0]) are de-asserted. Warning message is thrown out if any of the control signals in question are asserted when none of the banks' CS is asserted.

Configuration Parameters

The following parameters values are set via the configuration file

- Address Setup / Hold time
- Data Setup / Hold time
- Depth of Memory

9.2.3 TrickMem Register Description

The base address of the TrickMem is not fixed and may be different based on system implementations. However, the offset of any particular register from the base address is fixed.

SMCTrMEMR: SMC TrickMem Memory Array Register

Bit	Name	R/W	Reset value	Function
31- 0	MEMR	R/W	0x00	Purpose of this register is to provide read write accessibility to the memory. This represents 2k memory space of 32 bit wide memory. ***Figure 9.2-3*** shows this memory space.

Table 9.2-4 SmcTrickMem Memory array register

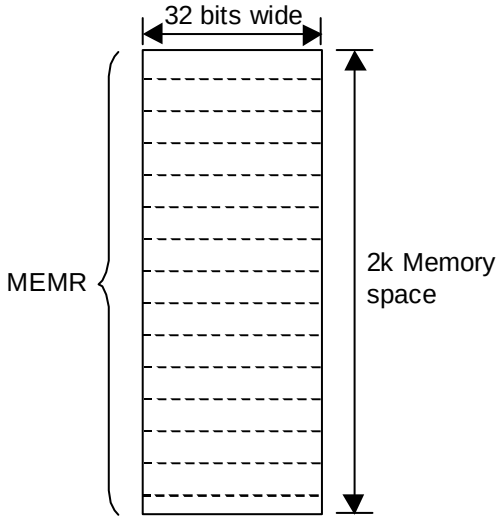

Figure 9.2-3 Memory space 32-bit wide

SMCTrIDCY: SMC TrickMem Idle Time Register

Bit	Name	R/W	Reset value	Function
4 – 0	IDCY	R/W	0x00	Memory data bus turn around time. The turn around time is (IDCY + 1) x T_{HCLK}

Table 9.2-5 SmcTrickMem Idle Time register

SMCTrWST1: SMC TrickMem Wait State1 Register

Bit	Name	R/W	Reset value	Function
5 – 0	WST1	R/W	0x00	In the case of SRAM and ROM, this field indicates read access time. In the case of Burst ROM this field indicates the initial access time. The wait state time is (WST1 + 1) x T_{HCLK}

Table 9.2-6 SmcTrickMem Wait state1 register

SMCTrWST2: SMC TrickMem Wait State2 Register

Bit	Name	R/W	Reset value	Function

5 – 0	WST2	R/W	0x00	In the case of SRAMs this field indicates the Write access time. The access time in this case is calculated as (WST2 + 1) x T_{HCLK} . In the case of Burst ROMs this field indicates the Burst Read access time. The access time in this case is calculated as (WST2) x T_{HCLK} . In the case of ROMs this field is insignificant.

Table 9.2-7 SmcTrickMem Wait state2 register

SMCTrCS2OEN: SMC TrickMem CS assertion to OEN assertion wait register

Bit	Name	R/W	Reset value	Function
4 – 0	CS2OEN	R/W	0x00	In the case of SRAMs and ROMs this register determines the time duration between Chip select assertion and the assertion of nSMOEN signal. In the case of Burst ROMs initial access this register determines the time duration between Chip select assertion and the assertion of nSMOEN signal and is insignificant in successive Burst ROM accesses.

Table 9.2-8 SmcTrickMem CS assertion to nSMOEN assertion register

SMCTrCS2WEN: SMC TrickMem CS assertion to WEN assertion wait register

Bit	Name	R/W	Reset value	Function
4 – 0	CS2WEN	R/W	0x00	In the case of SRAMs this register determines the time duration between Chip select assertion and the assertion of nSMWEN signal. In the case of Burst ROMs and ROMs this field is insignificant.

Table 9.2-9 SmcTrickMem CS assertion to nSMWEN assertion register

SMCTrMEMT: SMC TrickMem Memory Type Register

Bits	Name	R/W	Reset value	Function
1 – 0	MEMSIZE	R/W	0x00	MEMSIZE bits determine the width of the memory model simulated by the TrickMem. 0x00 for 8-bit wide memory 0x01 for 16-bit wide memory 0x02 for 32-bit wide memory

3 – 2	MEMTYPE	R/W	0x00	MEMTYPE bits determine the type of the memory model simulated by the TrickMem 0x00 for SRAM 0x01 for ROM 0x02 for BURST ROM
4	MaxAccErMask	R/W	0x00	1 = Error message disable 0 = Display error message if the read access time exceeded then the expected value and whether there is a bank to bank idle time violation or not.
5	AccErMask	R/W	0x00	1 = Error message disable 0 = Display error message if any violation happened in the read access time.
6	RBLE	R/W	0x00	0 = Memory banks configured as Write Enable controlled. 1 = Memory banks configured as Byte Enable controlled.
7	CSPolarity	R/W	0x00	0 = Memory banks respond when CS is active low 1 = Memory banks respond when CS is active high.
8	Reserved	R/W	0x0	
9	Reserved	R/W	0x0	
10	BoundaryCase	R/W	0x0	0 = The read-data will be returned by the TrickMem after WST1/WST2 +/- CEWT 1 = Irrespective of whenever the SMWAIT is de-asserted, data will always be returned after WST1/WST2.

Table 9.2-10 SmcTrickMem Memory Type register

SMCTrMEMB: SMC TrickMem Memory addresses base Register

Bit	Name	R/W	Reset value	Function
14 – 0	BASEVALUE	R/W	0x00	The TrickMem contains only 8K of memory. The SMC can address up to 64MB of memory. Using SMCTrMEMB register value as a base value and the 8K TrickMem internal memory as an offset it is possible to simulate 64-MB memory requirement for the test.

Table 9.2-11 SmcTrickMem Memory address base register

SMCTrCSPOL: SMC TrickMem Chip Select Polarity Register

Bit	Name	R/W	Reset value	Function

7 – 0	CSPOL	R/W	0x00	This register contains the Chip Select polarity of all the other banks. In the SMCTrMEMT register there is already a bit which determines the Chip Select Polarity of the memory connected to that particular bank. But for protocol checking, it is also required to know about the Chip Select Polarity of other banks. Hence this additional register.

Table 9.2-12 SmcTrickMem Chip Select Polarity register

10 TIC BLOCK VERIFICATION STRATEGY

The testing of the TIC module is done in the TICTALK world. During the TIC testing, the SmcCore module will be idle. The Ticbox module in the TICTALK world drives the TIC vectors onto the TIC module. The Ticbox reads the infile.tif (or tif.sim in the case of verilog version) and generates TIC vectors. A generic AHB slave is used for the testing of the TIC ***(Section 5.1)***. The TIC generates AHB transfers to the generic AHB slave. Different types of transfers are initiated and the behaviour of the TIC is tested. A TICWatcher module checks for the data integrity and protocol checks on the TIC generated AHB signals ***(Section 5.2)***.

10.1 Generic AHB Slave

The TIC is validated by using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

The block diagram of the Generic AHB Slave is shown in ***Figure 10.1-1***.

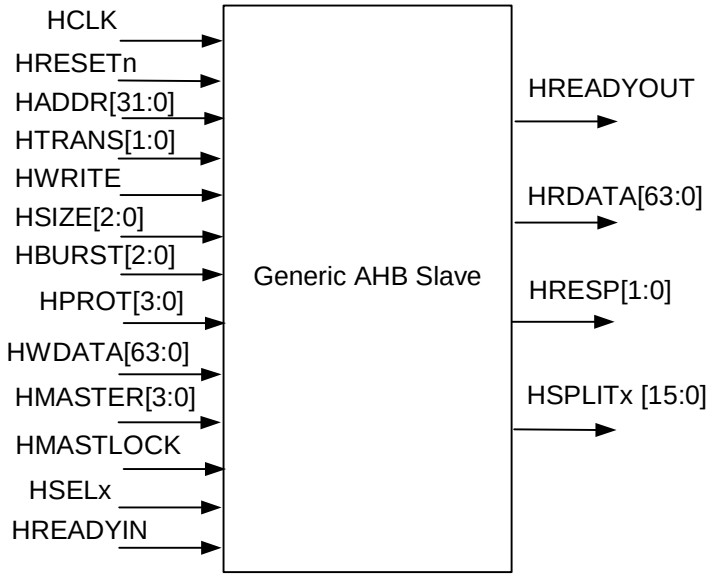

Figure 10.1-1 Generic AHB Slave Block diagram

10.1.1 Generic Slave Specifications

Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
Base Address + 0x00	R/W	32	0x0000	WSC1	Wait State Register 1 defines the wait states to be introduced while accessing memory array 1
Base Address + 0x04	R/W	32	0x0000	WSC2	Wait State Register 2 defines the wait states to be introduced while accessing memory array 2
Base Address + 0x08	R/W	32	0x0000	CR	Control Register defines DELAYEN bits for all output signals and PROTECTION bits for HRESP and HRDATA. Also defines the endianness of the system.
Base Address + 0x0C	R/W	32	0x0000	TMR	Timeout register defines the number of clock cycles after which the transfer times out in a SPLIT/RETRY transfer.
Base Address + 0x10	R/W	32	0x0000	XR	Transfer delay register determines when the slave has to respond to an access to a RETRY/SPLIT space with an OK response

Table 10.1-1 Functional Mode Registers

Memory Arrays

Address	Type	Width	Reset Value	Name	Description
Base Address + (0x100 – 0x4FF)	R/W	64	0x00	ARRAY1	Memory Array 1 to store data to check read and write operations
Base Address + (0x500 – 0x8FF)	R/W	64	0x00	ARRAY2	Memory Array 2 to store data to check read and write operations

Table 10.1-2 Memory Arrays

10.1.2 Generic Slave Description

The AHB slave will contain two register arrays and five configuration registers as per the following description. In the following all addresses are word addresses:

Memory Array

This is an array of 128 DWORDs (64 bits) which is accessible for reads or writes over the AHB. Aligned byte / half-word, word and DWORD accesses to this array of 1KB are supported with wait states controlled on a per-beat basis as per the Array Wait State Control Register. The array is divided into four parts such that accessing each part will give a different type of HRESP response.

Wait State Control Register

Any R/W access to a particular array location will introduce that many wait states as is programmed in the wait state register. The 32 bit wait state register is split into 8 fields with each field denoting the number of wait states to be introduced during an access to each beat of a burst transfer to that particular array. It is intended to do read/write operation to the register with zero wait states.

For a 4 word burst

bits 7:0 = wait states in beat 0

bits 15:8 = wait states in beat 1

bits 23:16 = wait states in beat 2

bits 31:24 = wait states in beat 3

For an 8 word burst

bits 3:0 = wait states in beat 0

bits 7:4 = wait states in beat 1

bits 11:8 = wait states in beat 2

bits 15:12 = wait states in beat 3

bits 19:16 = wait states in beat 4

bits 23:20 = wait states in beat 5

bits 27:24 = wait states in beat 6

bits 31:27 = wait states in beat 7

Control Register

The Control register contains 32 bits, of which 6 are used and the others are reserved. Each of the response signals generated depends upon the two programmable bits in the generic slave control register. These two bits help to find out whether the TIC is capable enough to detect the error in the responses given out by the generic slave.

Bit [0]: BIGENDIAN – This bit will determine whether the Slave will behave as little-endian or big-endian.

Bit [1]: ENDIANEN – When this bit is set, Slave will behave as a big endian system, driving the data only in the required byte lanes.

Bit [2]: Force Error Response output – On setting this bit the intention is to force errors by complementing the HRESP lines

Bit [3]: Forced Error Data output – On setting this bit the intention is to force errors by complementing the HRDATA lines

Bit [4]: RESPEN – This bit, if not set, will force the slave to drive the OK response. Otherwise, if set, slave is intended to behave normally.

Bit [5]: DELAYEN – If this bit is enabled, all the output signals are delayed for the default value programmed to check the timing protocols of the signal. This will help in checking the timing protocols of the signals.

Timeout Register

The 32 bit register determines when the slave will timeout for a retry/split transfer. After timeout, the slave won't proceed with the ongoing split/retry transfer.

Transfer Delay Register

The first 16 bits will determine on which re-issue of the SPLIT transfer, the generic slave has to respond with an OK response. The higher order 16 bits will determine the number of re-issues required in a retry transfer.

Transfer Modes

The generic slave supports almost all modes in AHB Slave architecture. The modes supported are listed below.

HTRANS[1:0] All combinations are supported

HSIZE [2:0] All transfers up to DWORD are supported

HBURST[2:0] All combinations are supported.

HRESP [1:0] All combinations are supported.

HPROT [3:0] Not supported.

10.2 TIC Watcher Module

The TIC Watcher module is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

10.2.1 TIC Watcher Signal Description

The following table summarises the TIC Watcher signal list.

Signal Name	Type	Source / Destination	Description
TCLK	IN	Clock Generator	Test mode clock input. This clock times all TIC bus transfers. All signal timings are referenced to the rising edge of TCLK.
RESETn	IN	Reset Controller	Bus reset (active low).
TESTREQA	IN	TICbox (External tester)	Test bus request A
TESTREQB	IN	TICbox (External tester)	Test bus request B
TESTACK	IN	TICbox (External tester)	Test Acknowledge
TESTBUS[31:0]	IN	TICbox (External tester)	Test data bus
HGRANTtic	IN	AHB Arbiter	This signal indicates that the TIC is currently the highest priority master. Ownership of the address/control signals changes at the end of the transfer when **HREADYIN** is HIGH.
HADDROUT[31:0]	IN	TIC	The 32-bit system address bus.

HTRANSOUT[1:0]	IN	TIC	Indicates the type of the current transfer, which can be Non-Sequential, Sequential or Idle. The TIC does not use the Busy transfer type.
HWRITEOUT	IN	TIC	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEOUT[2:0]	IN	TIC	Indicates the size of the transfer, which is typically byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.
HBURSTOUT[2:0]	IN	TIC	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTOUT[3:0]	IN	TIC	The Protection control signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a supervisor mode access or user mode access. These signals can also indicate whether the current access is cacheable or bufferable.
HWDATAOUT[31:0]	IN	TIC	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this may easily be extended to allow for higher bandwidth operation.

Table 10.2-1: The TIC Watcher Signal Description

10.2.2 TIC Watcher Functional Description

The TIC Watcher block diagram is shown in ***Figure 10.2-1***.

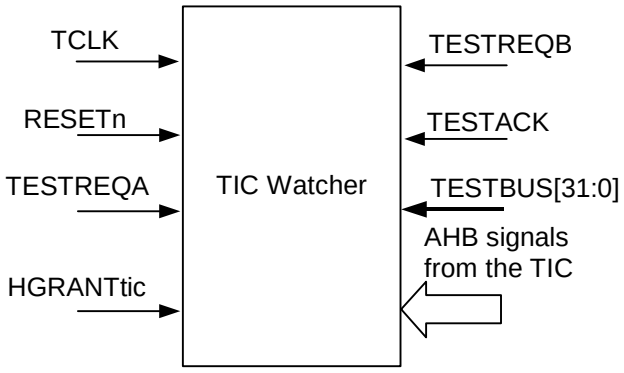

Figure 10.2-1 The TIC Watcher Block Diagram

The TIC Watcher receives the same vectors as the TIC does, from the Ticbox and generates AHB signals internally. It then compares the internally generated signals with the corresponding signals from the TIC and flags error messages whenever a protocol violation is detected on the bus.

11 FUNCTIONAL TESTS

The following sections describe all the test cases that validate the functionality of the SmcCore and TIC module. These test cases use the SmcCore Trickbox for their execution. For the validation of the TIC module,

SMCTrickTIC is used to drive the TIC vectors onto the TIC. The corresponding test code name is given in the parenthesis.

11.1 SmcCore Validation Test Cases

11.1.1 Register reset value tests [ResetTests.c]

The SMCBCR0, SMCBCR1, SMCBCR2, SMCBCR3, SMCBCR4, SMCBCR5, SMCBCR6, SMCBCR7, SMCBSR0, SMCBSR1, SMCBSR2, SMCBSR3, SMCBSR4, SMCBSR5, SMCBSR6, SMCBSR7, SMBEWS, SMCPeriphID0, SMCPeriphID1, SMCPeriphID2, SMCPeriphID3, SMCPCellID0, SMCPCellID1, SMCPCellID2 and SMCPCellID3 registers are read immediately after reset to verify that they initialised to the values mentioned in the Ref. 2.

11.1.2 Register read / write test [RegisterTests.c]

All writeable registers are written with patterns such as 0x00000000, 0x55555555, 0xAAAAAAAA and 0xFFFFFFFF. The data is read back from the registers and compared with the expected data. The expected data will depend on the register size and read/writeability of the register. The setting and clearing mechanism of the SMCBSRx registers will be checked during other memory transfer and SmcCore performance tests.

HSELREG is used for accessing the internal registers of the SmcCore. The internal register read-writes are additionally verified by asserting HSELSMC and ascertaining that the internal registers are not modified and vice-versa.

11.1.3 Address toggle test [AddrToggleTests.c]

This test toggles all memory-related address bits in the SmcCore. The TrickMem is designed with a memory size of 8K. The remaining higher order address is specified in the base address register SMCTrMEMB. So it is possible to access all 64MB addresses of the SMC by changing the value of SMCTrMEMB register.

11.1.4 Specified length burst test [TransferTests.c]

This is a generic full fledged test which does read-write operation to the memory device with all possible combinations of the following :

- HSIZE values of BYTE, HWRD and WRD
- Memory size of 8 bits, 16 bits and 32 bits
- Different HBURST values : SINGLE, INCR, WRAP4, INCR4, WRAP8, INCR8, WRAP16, INCR16
- Bank values of 0 to 7

The SmcCore does not support SPLIT or RETRY responses. For each transfer, the slave buswatcher of the top level testbench continuously monitors the slave response. Any deviation from the expected response will cause the error message to be flagged. Different tests use different values of read and write access values. Different test cases are tabulated below:

Transfer Sequences
NONSEQ
NONSEQ - NONSEQ to same bank
NONSEQ - NONSEQ to different banks

NONSEQ - NONSEQ - NONSEQ to same bank
NONSEQ - NONSEQ - NONSEQ to different banks
NONSEQ - SEQ - SEQ
NONSEQ - IDLE - NONSEQ to same bank
NONSEQ - IDLE - NONSEQ to different banks
NONSEQ – BUSY - SEQ
NONSEQ – BUSY - NONSEQ to same bank
NONSEQ – BUSY – NONSEQ to different banks
NONSEQ – BUSY – IDLE
NONSEQ - SEQ – BUSY – SEQ
NONSEQ – BUSY – SEQ - BUSY – SEQ
NONSEQ – BUSY – SEQ - BUSY
NONSEQ – BUSY - BUSY – SEQ
NONSEQ - SEQ - SEQ – SEQ - SEQ - SEQ - SEQ - SEQ
NSR-NSW-NSR to same and different banks
NSW-NSR-NSW to same and different banks
NSR-SR-NSW to same and different banks
NSW-SW-NSR to same and different banks

Table 11.1-1 Different Transfer Sequences

In the above table the following acronyms have been used :

NSR : NONSEQ Read

NSW : NONSEQ Write

SR : SEQ Read

SW : SEQ Write

The tests listed in the ***Table 11.1-1*** will be performed with the following different delays.

- Zero enable and wait delays
- One enable delay
- Two enable delays
- One wait state
- Two wait states
- One enable delay and two wait states
- Two enable delays and one wait state

- Non-zero turnaround cycles between transfers

All the transfers are verified by reading back the data from the AHB side of the Memory model. This ensures that the external address, control and data lines are properly driven out by the SMC.

Additional Corner Case Test :

1. A NONSEQ[WR] – IDLEs – NONSEQ[WR] transfer sequence is performed with the following conditions: An 8-bit write transfer is performed Bank-6 programmed as 8-bit followed by IDLE cycles for more than 1 clock cycles subsequently followed by a 8-bit write to Bank-3 configured as 16-bit. Finally these banks are read back to check the data integrity. Similar transfer sequence is performed with WORD writes to Bank-1 which is 32-bit followed by IDLE cycles followed by WORD write to Bank-3 programmed as 8-bit. This test is used to align some internal signals of the SMC to check the behaviour under specific internal conditions.
2. NONSEQ[WR] – IDLE – IDLE - ... – NONSEQ[WR] transfer sequence with the combinations as described. The first BYTE write to Bank-3 programmed as 16-bit followed by IDLE cycles equal to {WST2[Bank-3] + 2} followed by BYTE write to Bank-2 programmed as 32-bit. The two banks are read back to ascertain the data correctness. These types of test sequences are repeated for different banks with different HSIZE, MSize, WST2 and IDCY values. This test is used to align some internal signals of the SMC to check the behaviour under specific internal conditions.
3. Busy insertion at different positions in the burst – Transfer sequence with all HSIZE and all Burst transfers to 8-bit, 16-bit and 32-bit memory by inserting Busy at various positions.

11.1.5 Write Protection and Bus Error Flag tests [ProtectionTests.c]

This test checks the setting and clearing logic of the BUSERR and WPERR flags. The BUSERR flag is set when the master tries to perform an operation to the memory, with a transfer size greater than 32-bits. Writing ‘1’ to this bit location clears this flag. WPERR flag is set when the master tries to perform a write operation to a bank which is configured as ROM or Burst ROM. Writing ‘1’ to this bit location clears this flag. Tests are also performed for checking WPERR flag for write protection mode for SRAMs. In this mode a write operation is performed to a bank which is configured as SRAM with Write Protection bit set.

11.1.6 Multiple memory bank test [MemoryBankTests.c]

This test checks the operation of the SmcCore across the bank. The test sequence is as follows. Program the bank zero and the TrickMem connected to bank zero as SRAM of data size 32-bit. Program bank one and the TrickMem connected to bank one as SRAM of data size of 16-bit. Program bank three and the TrickMem connected to bank three as SRAM of data size of 8-bit. Program the different banks with different access times and different memory idle times. Random read and a write accesses to the memory from AHB with different addresses are performed such that it makes the SmcCore access different banks.

11.1.7 ROM test [ROMTests.c]

This test checks the operation of the SmcCore with different types of ROMs connected to different banks. The test sequence is as follows. Program bank zero, two, four, six and the TrickMems connected to these banks as ROMs with different WST1 values and different word sizes. Initialise the ROM contents from the AHB side in Configuration register and TrickMem. Read the memory banks through the SmcCore using different HSIZE and different HBURST values.

Additional Corner Case Test : This test emulates the scenario when IDLE transfers are introduced immediately after a read transaction with the following combinations HSELSMC=1, HWRITE=0 and Read has just completed (i.e. HREADYOUT has been given for the read transaction).

11.1.8 BURST ROM test [BROMTests.c]

This test checks the operation of the SmcCore with different types of BURST ROMs connected to different banks. The test sequence is as follows. Program bank one, three, five, seven and the TrickMems connected to these banks as BURST ROMs with different WST1, WST2 and word size values. Initialise the BURST ROM contents from the AHB side in Configuration register and the TrickMem. Perform burst and non-burst reads from memory banks through the SMC using different HSIZE and different HBURST values.

11.1.9 Chip select to Output Enable assertion and chip select to write Enable assertion test [DelayValueTests.c]

In these tests, different values are programmed in CS2OEN and CS2WEN registers and then burst reads, non-burst reads and writes are performed. All combinations mentioned below of read, write and burst read are tested.

- read followed by burst reads to same and different banks
- read followed by writes to same and different banks
- burst read followed by reads to same and different banks
- burst read followed by writes to same and different banks
- write followed by reads to same and different banks
- write followed by burst reads to same and different banks

11.1.10 Bank Crossing test [BankCrossingTests.c]

In these tests, various burst accesses of various lengths are attempted to the memory. The bursts are started at different word boundaries and quad word boundaries with INCR4, INCR8, INCR16, WRAP4, WRAP8 and WRAP16 for HSIZE of Byte, Halfword and Word such that these transactions cross over banks. The tests are repeated with various values of WST1, WST2, CS2OEN, CS2WEN and IDCY.

11.1.11 Endian-ness test [EndiannessTests.c]

The Endian-ness tests perform the following test sequences:

- Memory configured as 32 bits: Transactions are initiated from the AHB with HSIZE of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SmcCore.
- Memory configured as 16 bits: Transactions are initiated from the AHB with HSIZE of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SmcCore.
- Memory configured as 8 bits. Transactions are initiated from the AHB with HSIZE of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SmcCore.

The above 3 test sequences are repeated for all AHB burst types in little and Big Endian modes of operation where both reads and writes are of the same endianness. This test is repeated for sequences with little endian writes followed by big endian reads and big endian writes followed by little endian reads. Once Endian-ness is set, it is same for the whole system (AHB and Memory models).

11.1.12 Chip Select Test [CSTests.c]

Corresponding to any address, one and only one CS should be asserted. This test is performed to check this functionality. Also it checks whether the “chip selects” behave properly according to the polarity bit.

This is tested by applying all 8 values on HADDR[28:26], keeping remaining 26 address bits [25:0] unchanged. Eight memory locations are written into (such that [25:0] is same, but the chip-select changes), each with different data. The data is read back and compared. If it matches then unique CS assertion for each address is proved.

The test is repeated with different value of “POLARITY” passed to the function. Each CS polarity bit is made to change at least once.

11.1.13 SMWAIT tests [SMWAITTests.c]

The SMWAIT signal, when asserted, is supposed to ‘hold’ the memory accesses attempted by SmcCore. For the SmcCore, start of an access is signalled by assertion of Chip-Select or the change in SMADDR lines. The access finishes when a counter inside the UUT (the duration of count is controlled by the values programmed into the WST1/WST2 registers) expires.

To check this functionality, SMWAIT signal is asserted and de-asserted in various possible ways during an access. There are two registers in the TrickMem (SMCTrCEWT and SMCTrCS2WT) which help in controlling the time of assertion and de-assertion of SMWAIT, relative to Chip-Select and Counter Expiry, respectively.

The tests are repeated for both read and write accesses. These set of tests are performed with active low polarity of SMWAIT for the different banks i.e. with WaitPol = 0. The external wait signal polarity is programmable and the tests for checking the opposite polarity is carried out in the SMWAITFlagTests test code.

11.1.14 SMWAIT Flag tests [SMWAITFlagTests.c]

In these tests SMWAIT is asserted and de-asserted before and after 32 wait states and its effect on SMWAIT flag is monitored. SMWAIT is also asserted after WST1/WST2 followed by transfers to that bank and other banks to check SMWAIT flag is not set and it gives error responses. Boundary cases of assertion of SMWAIT just before the WST1/WST2 counter expiry and also on the same clock of counter expiry are also tested. A boundary case of de-assertion of SMWAIT on the 32 wait state is also tested.

In this set of test cases, active high polarity for the SMWAIT is used for the different bank i.e. WaitPol = 1. Since the external wait signal polarity is programmable, this ensures that the opposite polarity condition is also tested when compared to the tests in the SMWAITTests test code.

For the External Wait Timeout case, the status of the SMWAIT input is stored by the SmcCore in the SMBEWS read-only register, which can be polled by the software. This functionality is tested by asserting the CANCELSMWAIT input, when the SMWAIT input is already asserted. The setting of the Wait Time-Out Error flag is first checked and then the SMBEWS register is continuously polled to check if the SMWAIT status is reflected according to expectation. Also memory transfers are initiated from the AHB after the Wait Timeout Error flag is set, to check if error response is generated.

Additional Test :

1. This is an explicit test in which the SMWAIT is asserted exactly when the External Wait assertion counter reaches the expiry value. In this scenario it is expected that the SmcCore gives more priority to the SMWAIT assertion over the counter expiry.
2. A corner test case has been added, with the conditions satisfying the following specific sequence: An external wait controlled Write is initiated with HSIZE=32-bit to a memory device with MSize=16-bit. This transfer is aborted by triggering the SMWAIT time-out condition, by asserting the CANCELSMWAIT input. This is immediately followed by an AHB read transfer with HSIZE = 32-bit to a memory device of width 32-bit.

11.1.15 REMAP and SMMWCS7 tests [RemapTests.c]

In these tests REMAP is set to ‘0’ at Reset to check the functioning of SMC and also to check its effect on other chip selects and then REMAP is set to ‘1’ to operate in normal mode. The SMMWCS7 value, which is used to configure Bank 7 Memory size is varied for all possible combinations and simultaneously, checked for its

functionality. Initial values are loaded into the REMAP and SMMWCS7 registers after reset and then reset is applied again. So at second reset SMMWCS7 and REMAP have stable values.

11.1.16 Performance tests [PerformanceTests.c]

SmcCore performs reads, writes and burst reads to memory with different sizes and different banks. So this test is used to check whether the SmcCore gives data to the requester immediately. After completion of a valid read or write, the SmcCore can still hold HREADYOUT low. This test checks its performance by verifying that the read or write is completed within the expected number of clock cycles. The following timings are checked in this test:

- Wait State 1
- Wait State 2
- Idle/Turn-Around Cycles

Additional Corner Case Tests :

1. When a fresh write transfer (for which the HREADYOUT + OKAY response is given with zero waits) is in progress, next write transfer is initiated with the following condition. Between these two transfer requests, IDLE cycles equal to the WST2 (write access time value) are introduced. This condition is used to align the subsequent write request (kept pending on bus) exactly to the last cycle of the current write transaction.
2. In the above scenario instead of the subsequent write, a read with the same parameters is initiated.
3. A new/initial Write transfer is to be initiated when the SmcCore is performing turnarounds (i.e. it is in the TURN-AROUND state). Then the number of IDLE cycles required before the next write or a read transfer is requested is equal to the number of turn-around cycles remaining in addition to WST2 value. This condition is again used to align the subsequent write/read request (kept pending on bus) exactly to the last cycle of the current write transaction. With these specific conditions Write-Write and Write-Read transfer tests are performed.
4. Tests are done with write transaction to a bank having {a} WSTOEN=1 and WST2=1, and {b} WSTOEN and WSTWEN having same value other than ‘1’. This will test the SMWEN assertion delay logic and the access time counter logic for the equal values of the two parameters.

11.1.17 Random tests [RandomTests.c]

In these tests, various combinations of different sizes, different bursts and transfer types are attempted on different Banks. These accesses are generated randomly using a random function.

11.1.18 RBLE tests [RBLETests.c]

In these tests SmcCore is tested for RBLE functionality. In these tests we configure alternate banks with RBLE value ‘1’. Then various burst transfers are tried which crossover from one bank to next bank. These transfers are listed in the ***Table 11.1-1.***

Current Transfer with RBLE status	Next transfer with RBLE status
RBLE enabled read transfer	RBLE disabled write transfer to different bank
RBLE enabled write transfer	RBLE disabled read transfer to different bank
RBLE enabled read transfer	RBLE disabled read transfer to different bank
RBLE enabled write transfer	RBLE disabled write transfer to different bank
RBLE disabled read transfer	RBLE enabled write transfer to different bank

RBLE disabled read transfer	RBLE enabled read transfer to different bank
RBLE disabled write transfer	RBLE enabled read transfer to different bank
RBLE disabled write transfer	RBLE enabled write transfer to different bank
RBLE enabled write transfer	RBLE enabled read transfer to same bank
RBLE enabled read transfer	RBLE enabled write transfer to same bank
RBLE disabled write transfer	RBLE disabled read transfer to same bank
RBLE disabled read transfer	RBLE disabled write transfer to same bank

Table 11.1-2 Different RBLE transfers

In addition to the above conditions, the various combinations of RBLE enabled and disabled transfers are repeated with various different combinations of WST1, WST2, WSTOEN and WSTWEN. In order to hit certain corner cases, a specific number of IDLE/BUSY cycles are inserted between RBLE enabled and disabled accesses in order to ensure that certain internal conditions are hit in the SMC.

11.1.19 Busy and Idle accesses tests [BusAccessTests.c]

In these tests, SmcCore is tested by adding Busy Transfer in between reads, writes and burst reads. During continuous sequential Burst reads transfers, a BUSY can be inserted by the AHB master and then continue sequential Burst transfers. The SmcCore is expected to perform sequential Burst reads after the BUSY transfer (assuming the quad-boundary is not reached). On the other hand as SMC breaks burst transfer if an idle transfer come in between a burst transfer, so SMC is tested for this case by inserting idle transfers in between burst transfers. Idle transfers are also inserted in between reads and writes.

11.1.20 MC BUS Request-Grant Tests [MCReqGntTests.c]

In these tests, the MCBUSREQ signal is asserted in several possible ways. Different values are driven to the MCADDR, MCDATA and MCDATAEN lines using the configuration registers and READ/WRITE accesses to the MCBUSR address of the Trickbox. When the AMC (Additional Memory Controller) is granted, the SMC routes the MCADDR, MCDATA and MCDATAEN through the SMADDR, SMDATAOUT and nSMDATAEN lines respectively. The Trickbox keeps a watch on these lines and flags warning messages when a mismatch is found on the lines. Different cases tested are given below:

- MCBUSREQ is raised before the SmcCore requests for the BUS.
- MCBUSREQ is raised when the SmcCore is holding the BUS.
- MCBUSREQ is raised simultaneously as the SmcCore requests for the BUS.
- MCBUSREQ is raised when the SmcCore is doing a burst on the BUS.
- MCBUSREQ is raised when the SmcCore is in the IDLE state.

In each case, the memory transfer is verified by reading out the data from the memory.

Additional Corner Case Test : The MCBUSREQ is asserted when the SmcCore is performing turnaround cycles in between two read transactions to different banks i.e. (SmcCore is in TurnAround state). The WST1, IDCY delays are programmed appropriately to simulate this case.

11.1.21 Memory Model (TrickMem) Protocol Checks

The TrickMem contains protocol checks described in ***Section 4.2.3***. Along with these protocols, checks are done for the presence of ‘X’ on the following signals –SMADDR, SMCS, nSMDATAEN, nSMWEN, nSMOEN, nSMBLS.

11.1.22 Burst ROM Busy Insertion Test [BROMBusyInsertionTest.c]

In this test, reads and writes are done to burst mode enabled devices with BUSY transfers inserted in the burst. The starting addresses of the AHB bursts are selected randomly. Therefore both aligned and unaligned starting addresses are covered. The test is repeated for different values of HSIZE, Memory width and HBURST. This test also includes read and write fixed length AHB burst sequences that are terminated before the burst completes. The SMC is tested with the burst termination happening at a SEQ access or at a BUSY transfer.

11.1.23 Burst ROM Delay Test [BROMDelayTest.c]

The behaviour of the SMC for zero value of WST2 is tested in this test. A long AHB burst of unspecified length is initiated to a burst ROM with WST1 programmed as a non-zero value and WST2 = 0 to verify the functionality of zero-wait stated page mode reads. An additional case where a WRAP 4 read transfer is started at unaligned start address targeting a BROM is also included in this test. This case checks whether the quad-boundary detection logic functions correctly for WRAP transfers.

11.1.24 Denali Memory Model based Tests [DenaliTests.c]

The SmcCore performance is verified by the Denali memory models which emulates real world memory device. Transfers are done to the memory with different values of HBURST, HSIZE and HTRANS. Write protection tests are done by writing to ROM models. Corresponding to every transfer, the status registers (SMBSRx) are read for the updated status of the device. REMAP line is toggled and the mapping of the memory bank 7 to Bank 0 is verified. The memory models used for the simulation are given below. The SOMA files mentioned are in the *verification/denali* directory.

Sl. No.	SOMA File name	Datasheet used
1.	km68v1002c_12.soma	KM681002C-12, 128K x 8 High speed SRAM (Samsung).
2.	km68v1002c_20.soma	KM681002C-20, 128K x 8 High speed SRAM (Samsung).
3.	x28f640b3b_3v_100.soma	28F640B3B -3V-100 64 Mbit, 3 Volt Advanced Boot Block Flash Memory (Intel).
4.	x28f800c3b_110.soma	28F800C3B-100 3-Volt Strata Flash Memory (Intel).
5.	k3p9vu1000m_33v_yc.soma	K3P9V(U)1000M-3.3V_YC 128Mbit (16Mx8/8Mx16) CMOS Mask ROM (Samsung).
6.	sst37vf020_90.soma	SST37VF020-90 256x8 Many Time Programmable Flash (SST).
7.	km68v1002c_12.soma	KM681002C-12, 128K x 8 High speed SRAM (Samsung).
8.	km68v1002c_20.soma	KM681002C-20, 128K x 8 High speed SRAM (Samsung).

Table 11.1-3 Details of datasheets used for Denali Simulations using tb_Denali.v/vhd

11.1.25 Busy Insertion corner case [BusyInsCornerCase.c]

This test checks if the write with HTRANS = BUSY does not happen. Bank 0 is programmed as a R/W memory. A sequence is written to the memory (WR1) with no busy insertions {NSEQ, SEQ, SEQ, SEQ}. Then a sequence is written (WR2) with one NSEQ and rest all BUSY i.e. {NSEQ, BUSY, BUSY}. The consequent read checks that only the data which was written with NSEQ (WR2) is modified and rest all are same as WR1.

11.1.26 BROM Corner Case Test [BROMCornerCase.c]

The test uses TrickMem. Initially all the banks are written with an incremental data using Smc with burst mode and write protect de-asserted. Then Bank 0, 1 and 2 are programmed as BROMs of width 8, 16 and 32 bits respectively. They are read one by one. All combinations of following parameters are tested.

- All types of HBURST (SINGLE, INCR, INCR4, INCR8, INCR16, WRAP4, WRAP8, WRAP16)
- All types of HSIZE (BYTE, HWRD, WRD)
- Address at different quad word boundaries.
- Randomly generated sequence of HTRANS with types IDLE, NSEQ, SEQ and BUSY.

11.1.27 Big Endian HBURST Test [BigEndHburstTest.c]

In this test all the banks are programmed with different values of width, WST1, WST2, WSTOEN and WSTWEN. The system is programmed for Big Endian mode. The accesses are done with all combinations of following for write and read accesses:

- All types of HBURST (SINGLE, INCR, INCR4, INCR8, INCR16, WRAP4, WRAP8, WRAP16)
- All types of HSIZE (BYTE, HWRD, WRD)
- Address at different quad word boundaries.
- Randomly generated sequence of HTRANS with types IDLE, NSEQ, SEQ and BUSY.

11.1.28 Delay value ROM tests [DelayValueROMTests.c]

In this test 3 banks are configured as BROM of width 8, 16 and 32 bits, 3 banks are configured as ROM of 8,16 and 32 bits and remaining 2 as 8 bit and 32 bit SRAM (32-bit with RBLE).

With combinations of different values of WST1, WST2, WSTOEN a sequence of ROM/BROM access (burst) with SRAM access is followed. The focus of the test is testing different WSTOEN values with a set of WST1 and WST2 values for the various access combinations.

Following access combinations are tested.

- ROM read – BROM read – SRAM write,
- BROM read – ROM read – SRAM read,
- ROM read – BROM read – SRAM read,
- BROM read – ROM read – SRAM write,
- SRAM write – BROM read – SRAM read - BROM read.

Consecutive burst accesses are sequenced such that they are always to memories with different width. ROM /BROM accesses cover all hsize-msize combinations. Bursts always start at non burst aligned boundaries. BUSY transfers are inserted between the bursts, at burst address boundaries and at end of burst.

11.1.29 Mixed access tests [MixedAccessTests.c]

In this test all the memory banks are configured as SRAM (writeable) of various widths. The accesses are made such that address crosses over the bank boundary.

The accesses are done with all combinations of following for write and read accesses:

- All types of HBURST (SINGLE, INCR, INCR4, INCR8, INCR16, WRAP4, WRAP8, WRAP16)

- All types of HSIZE (BYTE, HWRD, WRD)
- Address at different quad word boundaries.

Various values corner values of WST1, WST2, WSTOEN WSTWEN and IDCY are tested.

11.1.30 Corner Case 1 [Cornercase1.c]

This test addresses the corner cases related to burst mode reads followed by writes to burst mode enabled SRAM devices. This test does different types of burst reads followed by a single write and read to the same bank. It performs burst reads from the memory banks through the SMC using different HSIZE, WST1, WST2 and HBURST values.

11.1.31 Corner Case 2 [Cornercase2.c]

In this test, a burst of read accesses are done to a burst mode enabled SRAM device and then followed by a single write and read to a different bank. It performs burst reads to the memory banks using different HSIZE, WST1, WST2 and HBURST values.

11.1.32 Corner Case 3 [Cornercase3.c]

In this test, a WRAP8 write with HSIZE < MSIZE is initiated to a bank that is programmed for large write access time. The WRAP8 write is then followed by some register accesses and then another AHB slave (the trickmemory is used in this case) is accessed with some BUSY transfers inserted in the burst. These accesses are then followed by an INCR16 write burst with HSIZE < MSIZE. This test ensures that busy transfers that are targeted to other AHB slaves do not affect the functioning of the SMC.

11.1.33 Corner Case 4 [Cornercase4.c]

In this test, a WRAP4 write to one bank is followed by a WRAP8 write to a different bank after a specific number of accesses to a different AHB slave. It is ensured that the first write of the WRAP8 is started on the clock where the last write in the WRAP4 write has ended on the memory bus.

11.1.34 Corner Case 5 [Cornercase5.c]

This test addresses corner cases related to accesses to various locations under various conditions of previous access. It checks if the performance of Smc for a certain transfer is hampered by the previous transfers. It involves accesses at quad word boundary, followed by an access to some other address, write to a register and checking if the new value is used immediately as the control value for next transfers (e.g. WST1_ini = 1, one read WST1_new = 2, next read and check if the WST1 value used was WST1_new) and such other sequences.

11.1.35 Corner Case 6 [Cornercase6.c]

This test addresses directed corner cases related to accesses to BROM. Successive broken bursts, access to registers and access to non BROM after/ before bursts of BROM reads are tested in this test. Accesses are made to different chipselects with different RBLE combinations, with various combinations of write, read, busy and idle. Different types of burst with different combinations of memory width and HSIZE are also tested. MCBUSREQ and SMBUSREQ combinations are also tested.

11.1.36 Corner Case 7 [Cornercase7.c]

In this test an access is made to a wait enabled bank and SMWAIT signal is made active before the turnaround for the access happens but after its wait state counters have expired.

11.1.37 Corner Case 8 [Cornercase8.c]

This test does a memory write followed by a memory read such that during read happens on memory side, CANCEL SMWAIT is given for the transfer.

11.1.38 Corner Case 9 [Cornercase9.c]

In this test, during a Memory read CANCELSMWAIT is given and further accesses are initiated on the AHB side with SMWAIT still asserted.

11.1.39 Corner Case 10 [Cornercase10.c]

This test case is directed towards CANCELSMWAIT behaviour. CANCELSMWAIT is given during a memory access and further accesses are done after SMWAIT is deasserted before turnaround for first access is done.

11.1.40 Corner Case 11 [Cornercase11.c]

Register access after a write to a wait enable bank is tested. A register access is done after a write to a wait enabled bank and HREADY behaviour is observed.

11.1.41 Corner Case 12 [Cornercase12.c]

In this testcase, a write is done to a non wait enabled bank followed by write to a wait enabled bank.

11.1.42 Corner Case 14 [Cornercase14.c]

This test is directed toward breaking the burst ROM accesses with external agent requesting for bus through MCBUSREQ. During a burst mode read, MCBUSREQ is given due to which SMBUSGNT will be removed after the current transfer.

11.1.43 Write Protect Error Test [WPErrorTest.c]

This test checks if writable memory writes followed by write protected memory writes generate the write protect error.

11.1.44 Access in Clock just after Reset [OneClkAfterReset.c]

Following access are tried in the clock after reset.

- Register write, memory write (memory read to check if write has happened)
- Register read (expecting Reset value), memory write (memory read to check if write has happened)
- Memory write, (memory read to check if write has happened)
- Memory read (expecting Reset value.)

11.1.45 NewRandomTest.c

- Configures all the memory banks with random control values.
- Access these banks for reading writing randomly with random burst sizes and transfer types.
- SMWAIT, CANCELSMWAIT and EBI signals are generated randomly.
- Does random register accesses in between memory access.

- A C function srandom is used to initialise function random w.r.t to a given seed value for the ease of re-creation of the scenario.

Seed value is taken from Seed.h file.

11.1.46 EbiCornercase.c

This testcase is directed towards testing the behaviour of EBI signals of the SMC when the EXTBUSMUX=1, i.e when external EBI master gives grant for the memory bus.

- EXTBUSMUX is driven high by trickbox.
- Memory Bank 2 is programmed to be of 8 bit wide.
- Trickbox is programmed to give the EBI grant in One clock.
- Burst writes and reads are done to Memory Bank 2 with all combinations of HSIZE,Burst type combinations.

11.2 TIC Validation Test Cases

TIC validation tests are done in the TICTALK world. All the tests mentioned below have been tested with different combinations of the control register bits.

11.2.1 Transfer tests [TransferTests.c]

In this test, vectors are applied in different sequences with different HSIZE values. Tests are done in Address Increment enabled/disabled mode of the TIC. Address boundary check is done by applying writes and reads to the boundary address in the address increment enabled mode.

11.2.2 Response tests [ResponseTests.c]

In this test, the generic slave is configured to give out different responses (ERROR/SPLIT/RETRY) and the TIC performance is tested.